

Intro to Neural Networks and Deep Learning

Jack Lanchantin
Dr. Yanjun Qi

UVA CS 6316

Neurons

1-Layer Neural Network

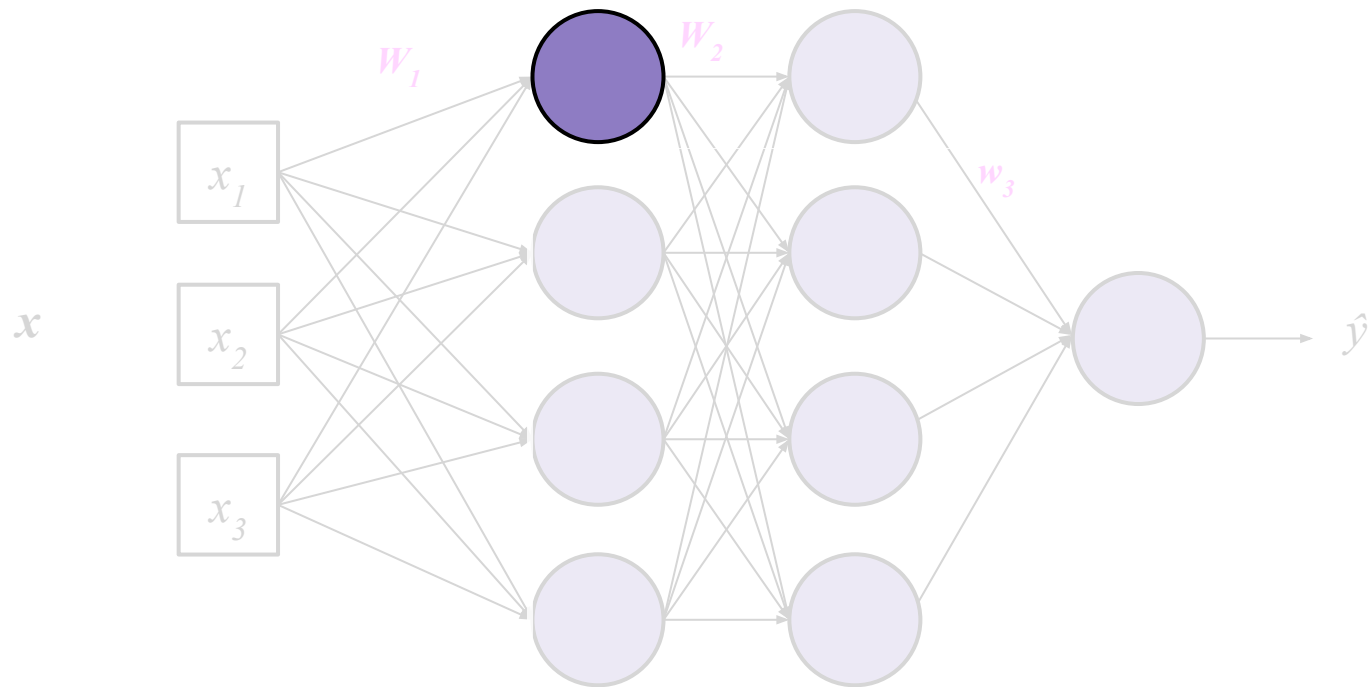
Multi-layer Neural Network

Loss Functions

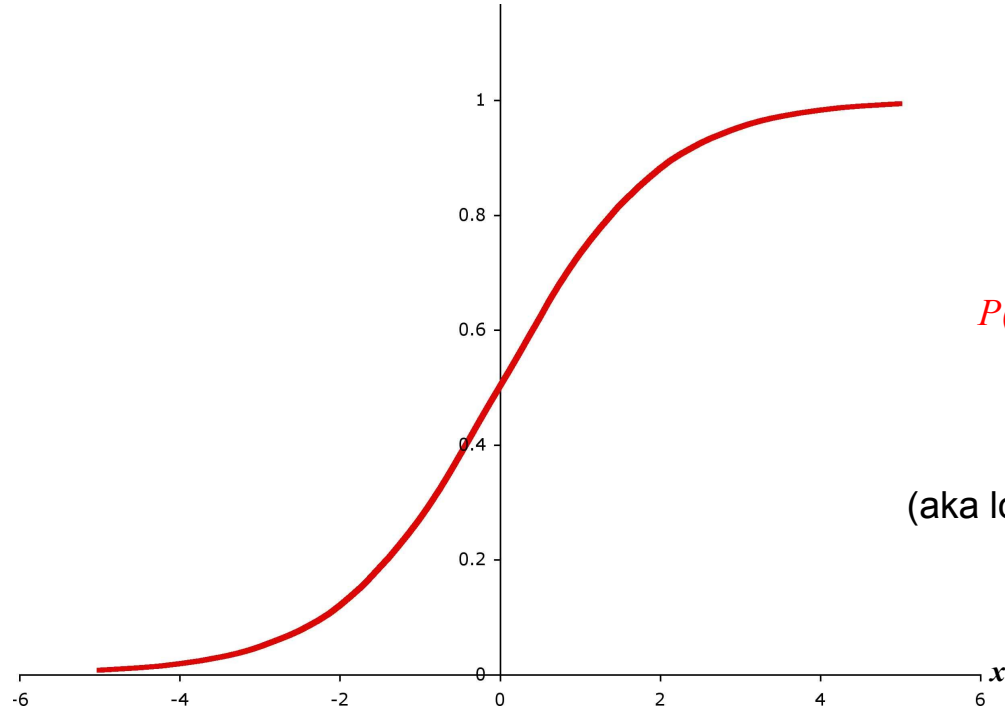
Backpropagation

Nonlinearity Functions

NNs in Practice



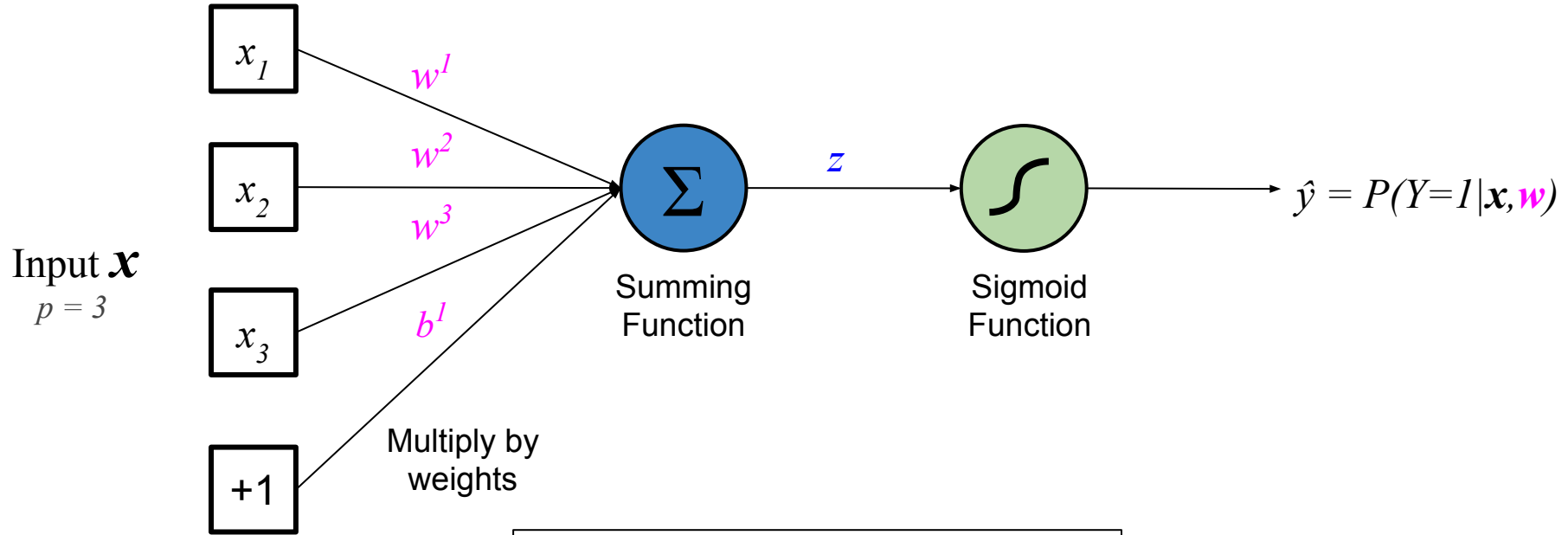
Logistic Regression



$$P(Y=1|\mathbf{x}) = \frac{e^{w\mathbf{x}+b}}{1 + e^{w\mathbf{x}+b}}$$

Sigmoid Function
(aka logistic, logit, “S”, soft-step)

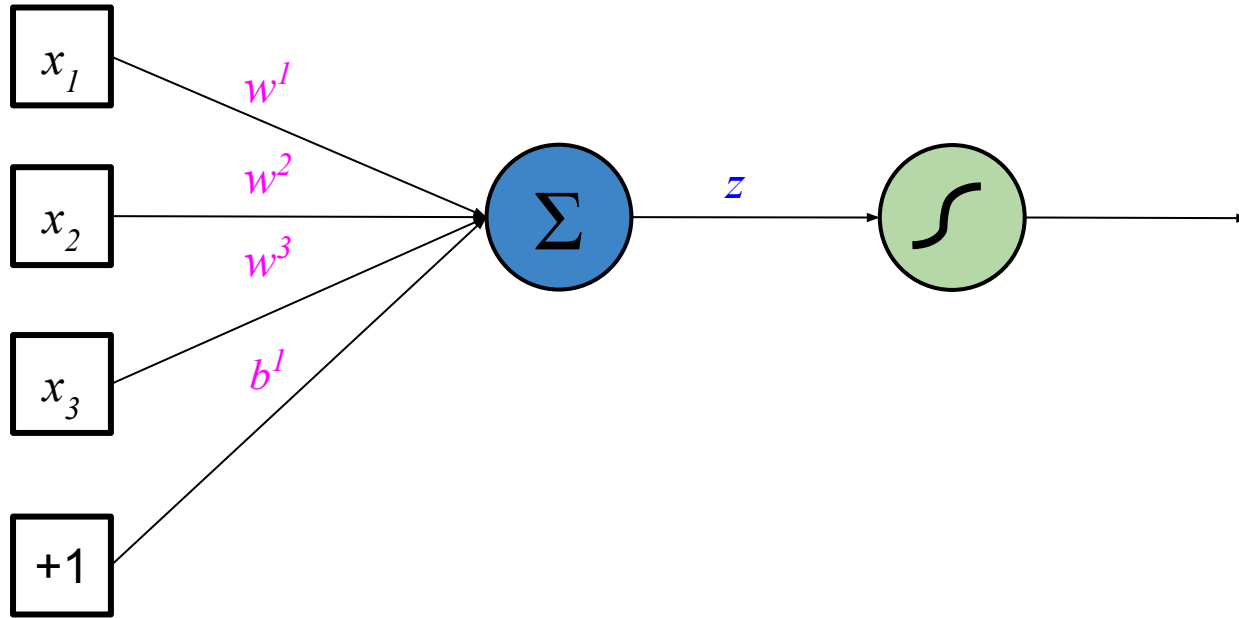
Expanded Logistic Regression



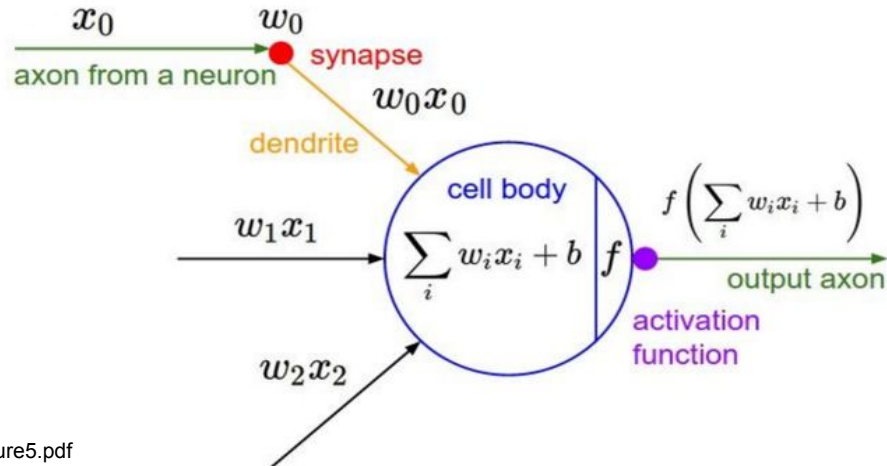
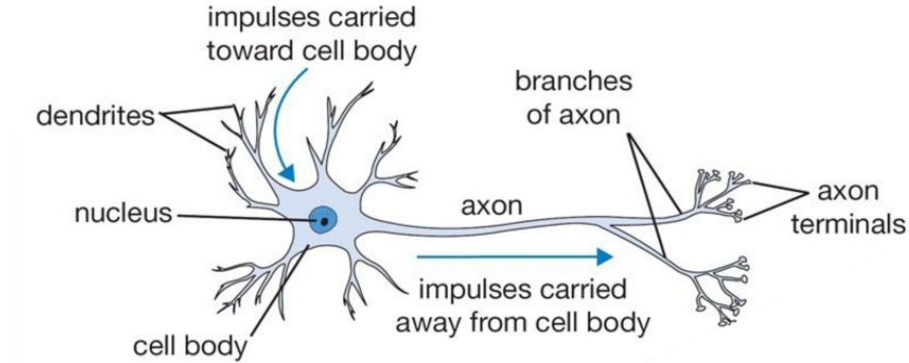
$$\underset{1 \times 1}{\mathbf{z}} = \underset{1 \times p}{\mathbf{w}^T} \underset{p \times 1}{\mathbf{x}} + \underset{1 \times 1}{b}$$

$$y = \text{sigmoid}(\mathbf{z}) = \frac{e^{\mathbf{z}}}{1 + e^{\mathbf{z}}}$$

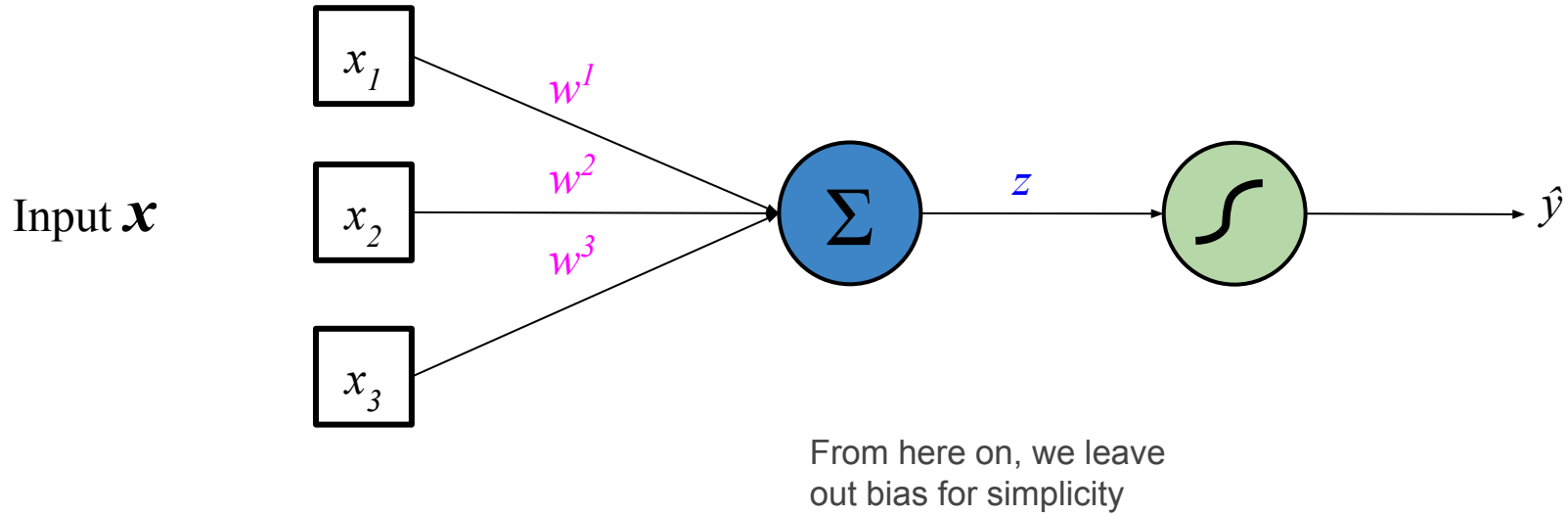
“Neuron”



Neurons



Neuron

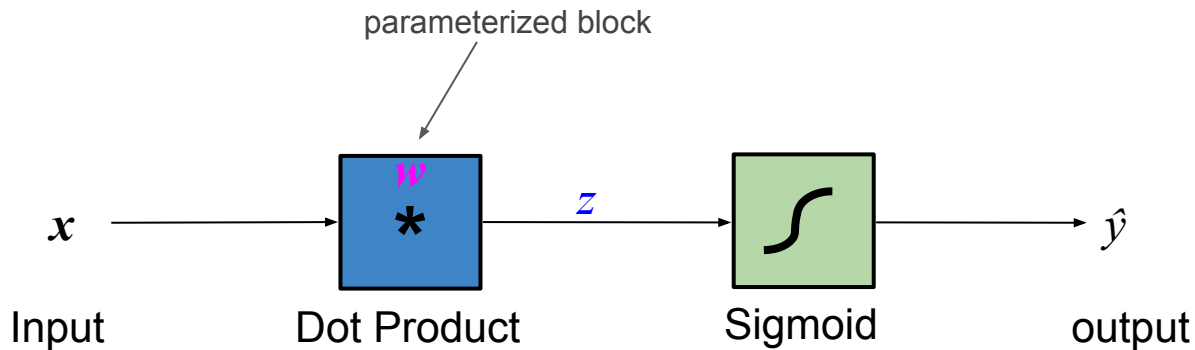


From here on, we leave out bias for simplicity

$$\underset{1 \times 1}{\mathbf{z}} = \underset{1 \times p}{\mathbf{w}^T} \underset{p \times 1}{\mathbf{x}}$$

$$\hat{y} = \text{sigmoid}(\mathbf{z}) = \frac{e^z}{1 + e^z}$$

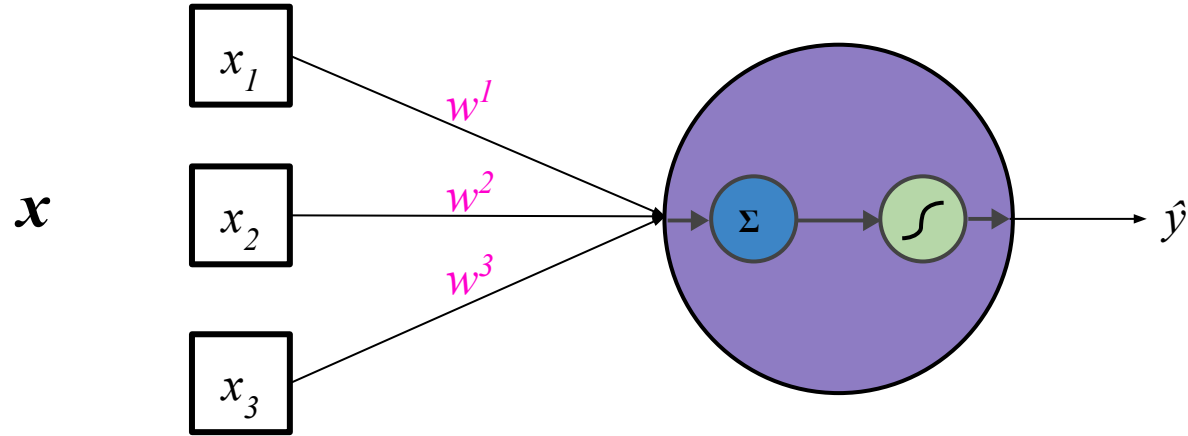
“Block View” of a Neuron



$$\underset{1 \times 1}{z} = \underset{1 \times p}{w^T} \underset{p \times 1}{x}$$

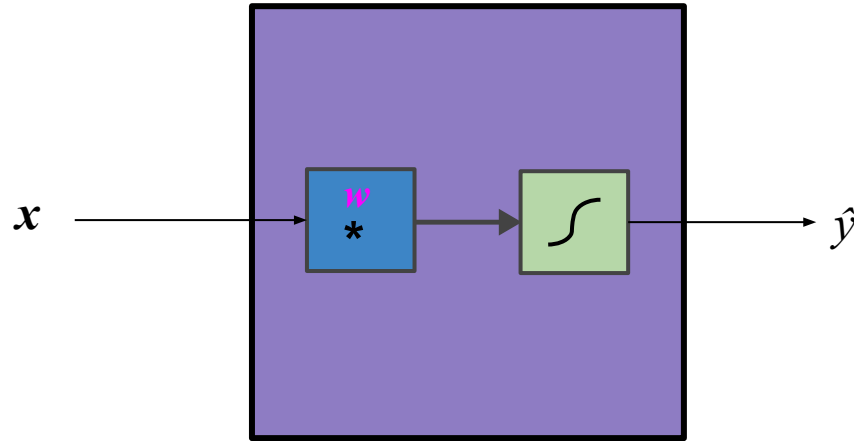
$$\hat{y} = \text{sigmoid}(z) = \frac{e^z}{1 + e^z}$$

Neuron Representation



The linear transformation and nonlinearity together is typically considered a single neuron

Neuron Representation



The linear transformation and nonlinearity together is typically considered a single neuron

Neurons

1-Layer Neural Network

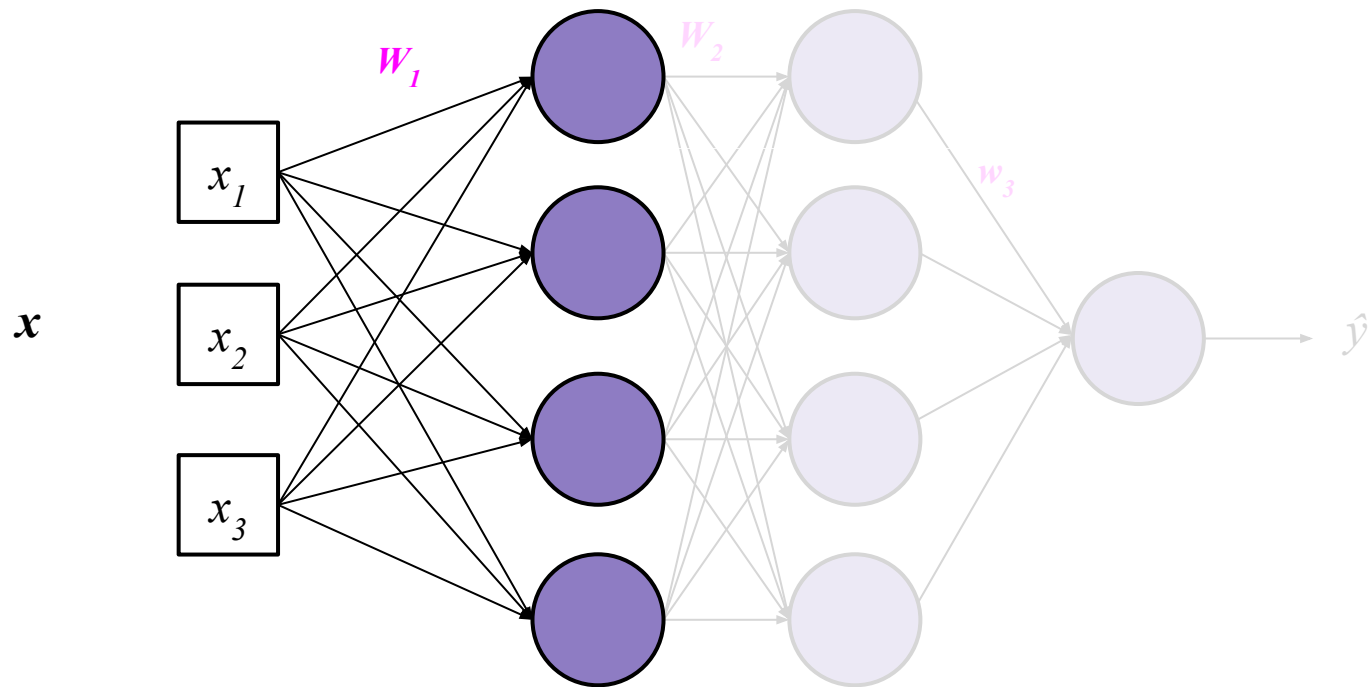
Multi-layer Neural Network

Loss Functions

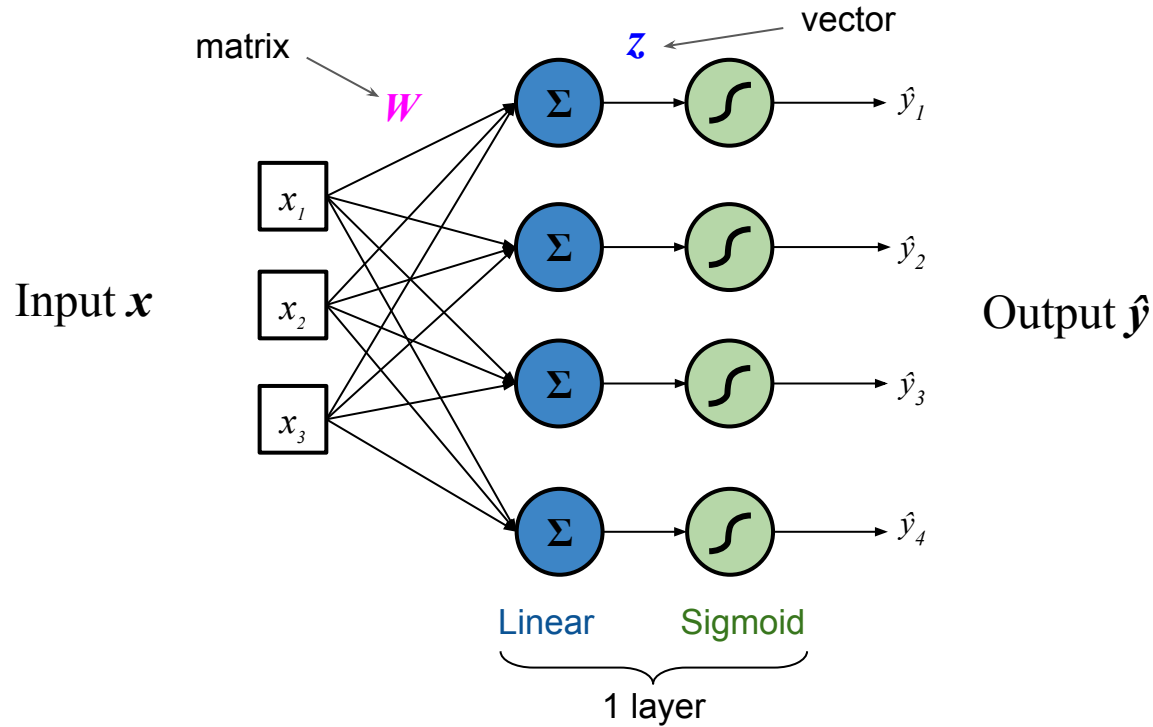
Backpropagation

Nonlinearity Functions

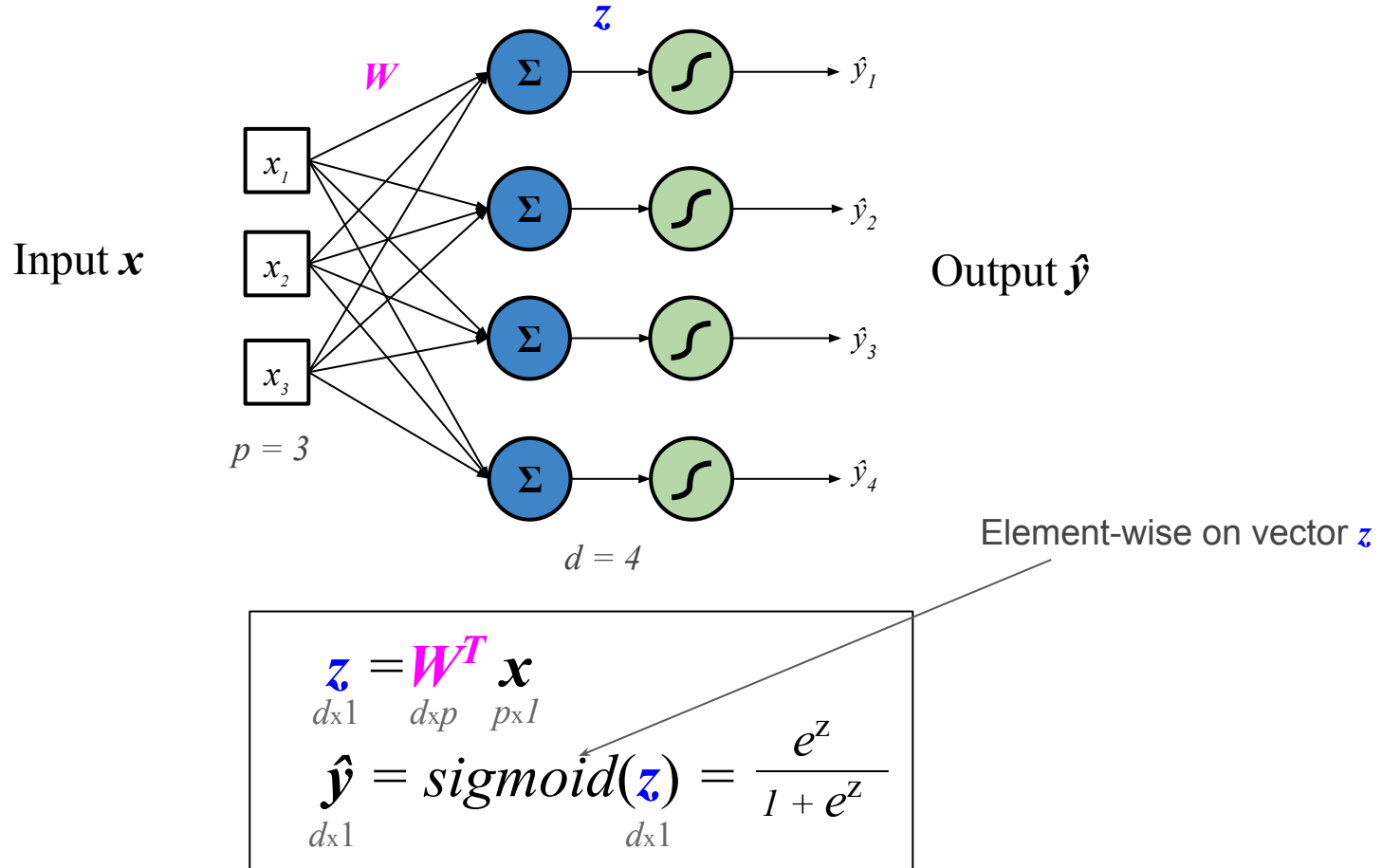
NNs in Practice



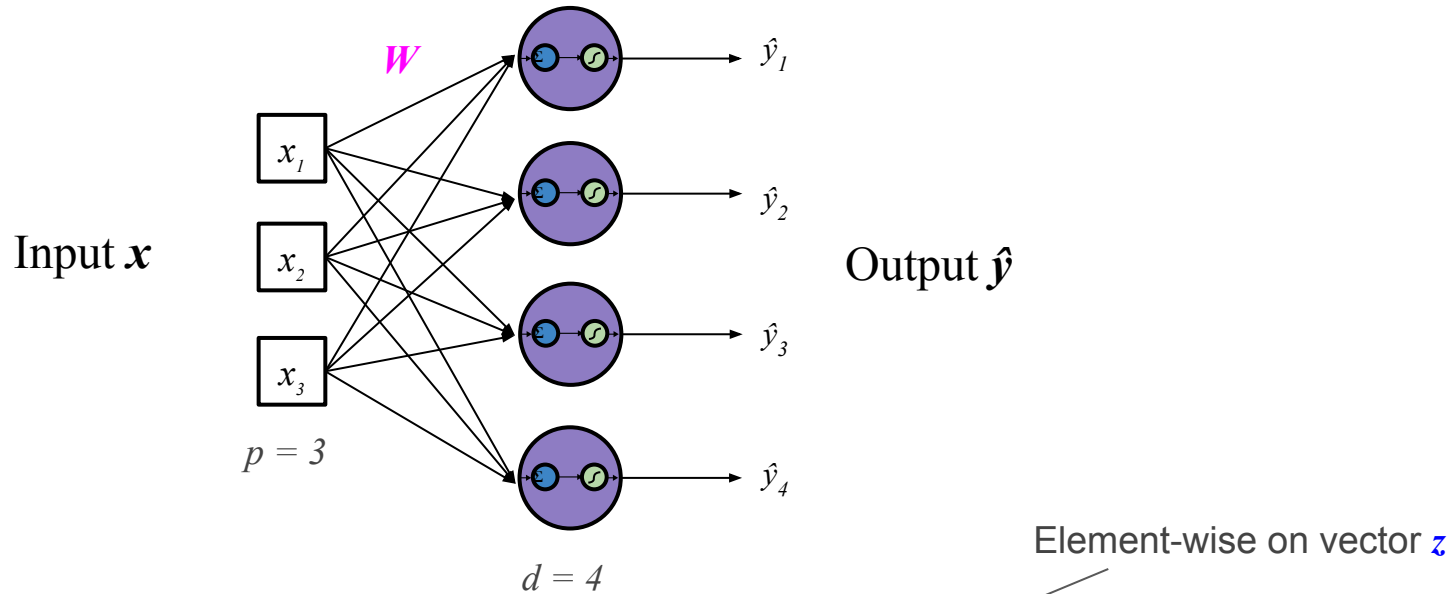
1-Layer Neural Network (with 4 neurons)



1-Layer Neural Network (with 4 neurons)

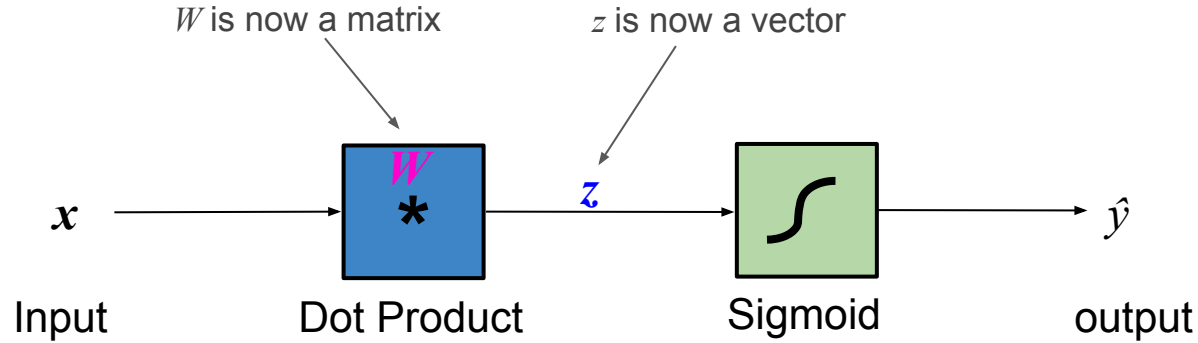


1-Layer Neural Network (with 4 neurons)



$$\begin{aligned} \underset{d \times 1}{\mathbf{z}} &= \underset{d \times p}{W}^T \underset{p \times 1}{\mathbf{x}} \\ \underset{d \times 1}{\hat{\mathbf{y}}} &= \underset{d \times 1}{\text{sigmoid}(\mathbf{z})} = \frac{e^{\mathbf{z}}}{1 + e^{\mathbf{z}}} \end{aligned}$$

“Block View” of a Neural Network



$$\underset{d \times 1}{z} = \underset{d \times p}{W^T} \underset{p \times 1}{x}$$
$$\underset{d \times 1}{\hat{y}} = \underset{d \times 1}{sigmoid}(z) = \frac{e^z}{1 + e^z}$$

Neurons

1-Layer Neural Network

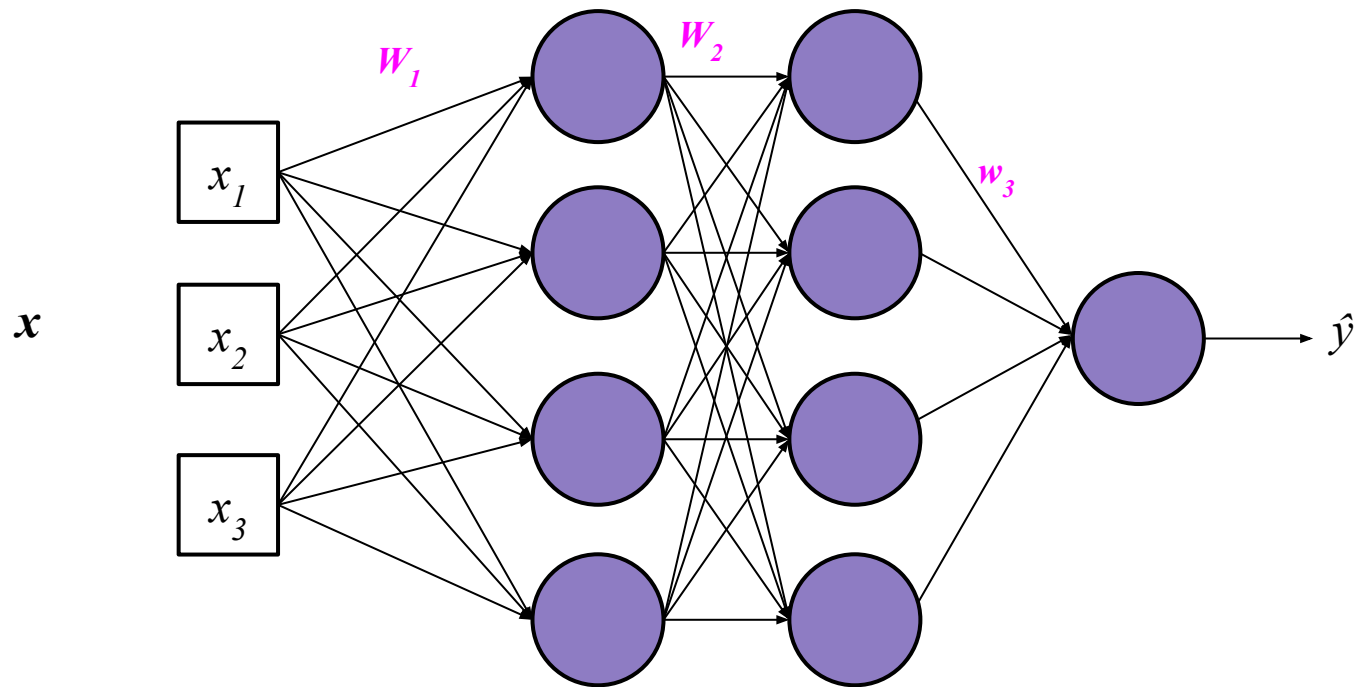
Multi-layer Neural Network

Loss Functions

Backpropagation

Nonlinearity Functions

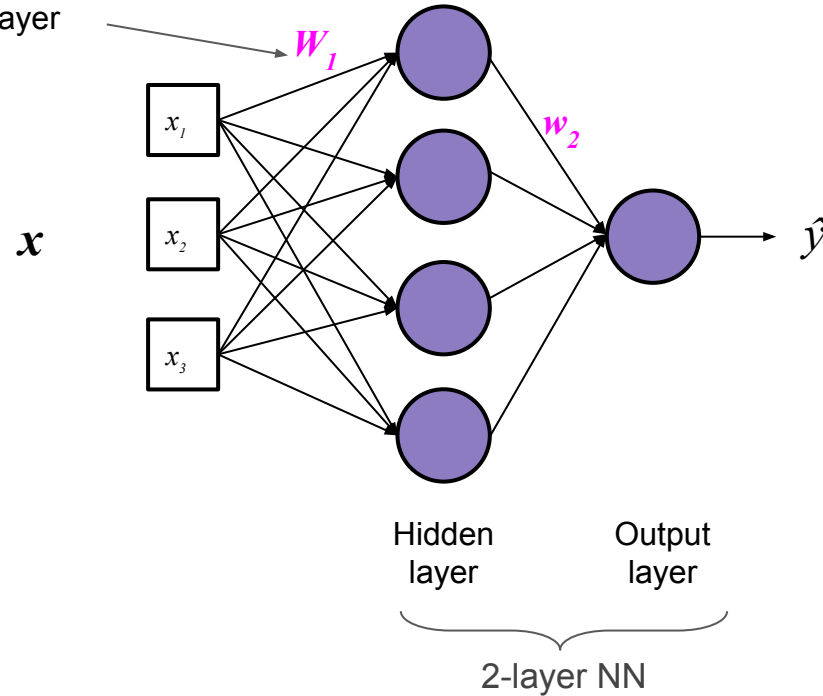
NNs in Practice



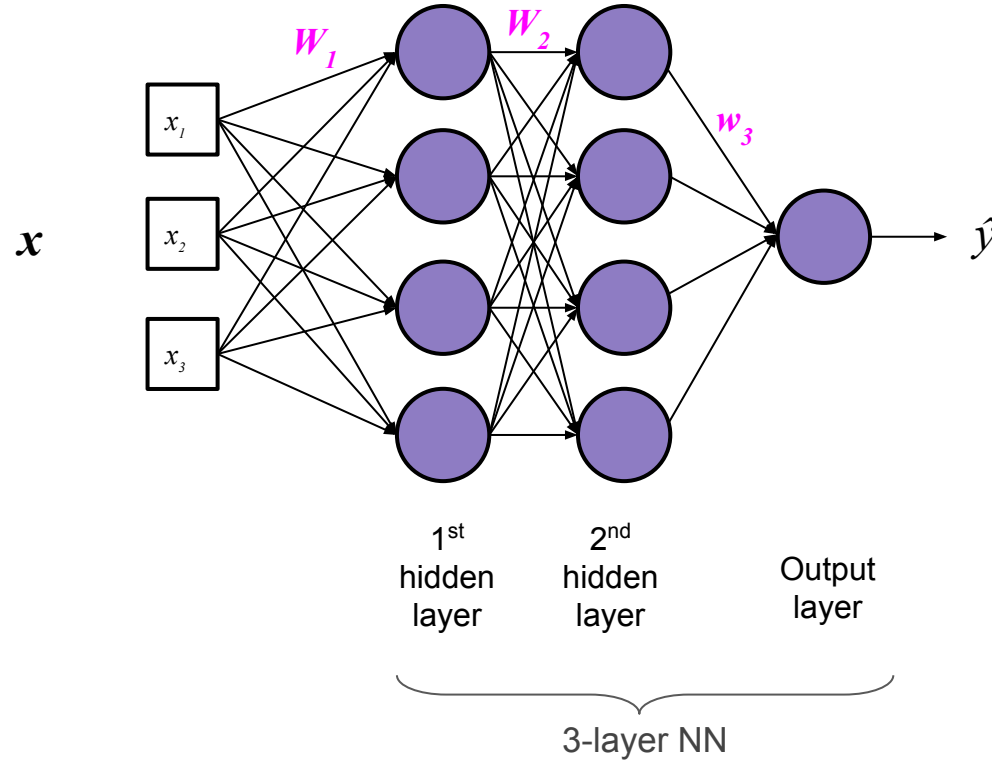
Multi-Layer Neural Network

(Multi-Layer Perceptron (**MLP**) Network)

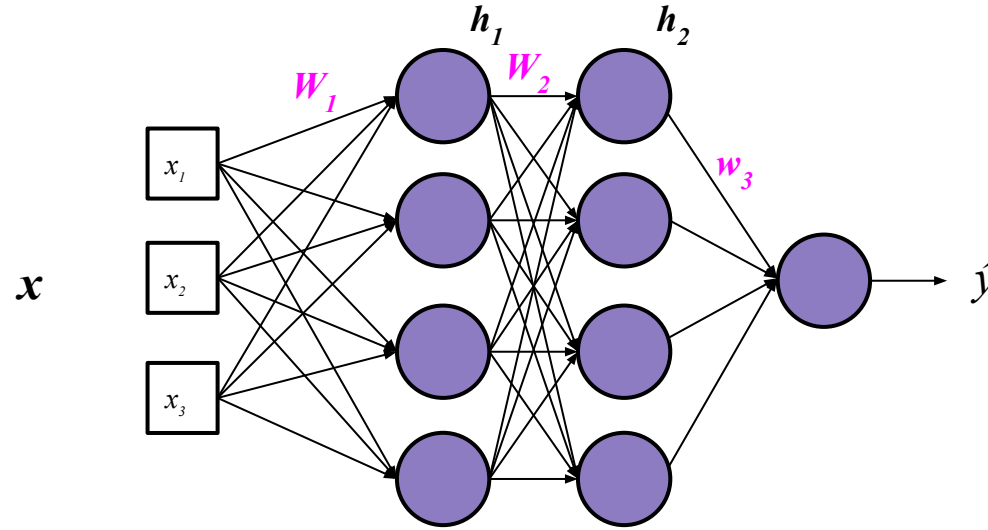
weight subscript
represents layer
number



Multi-Layer Neural Network (MLP)



Multi-Layer Neural Network (MLP)



hidden layer 1 output

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\mathbf{h}_1 = \text{sigmoid}(\mathbf{z}_1)$$

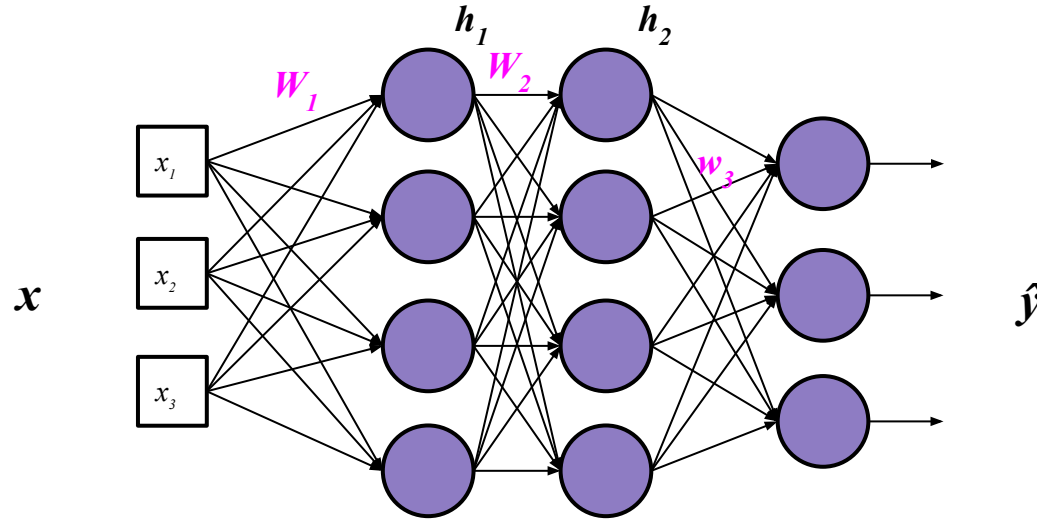
$$\mathbf{z}_2 = \mathbf{W}_2^T \mathbf{h}_1$$

$$\mathbf{h}_2 = \text{sigmoid}(\mathbf{z}_2)$$

$$\mathbf{z}_3 = \mathbf{w}_3^T \mathbf{h}_2$$

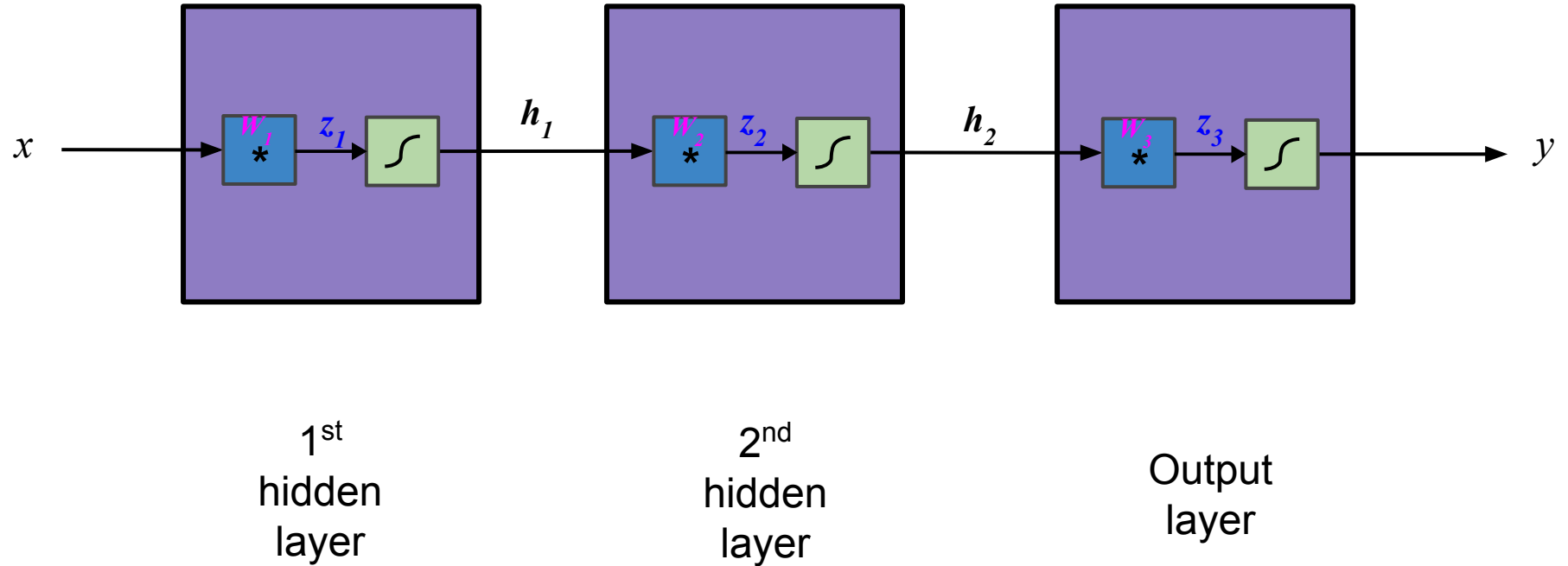
$$\hat{y} = \text{sigmoid}(\mathbf{z}_3)$$

Multi-Class Output MLP

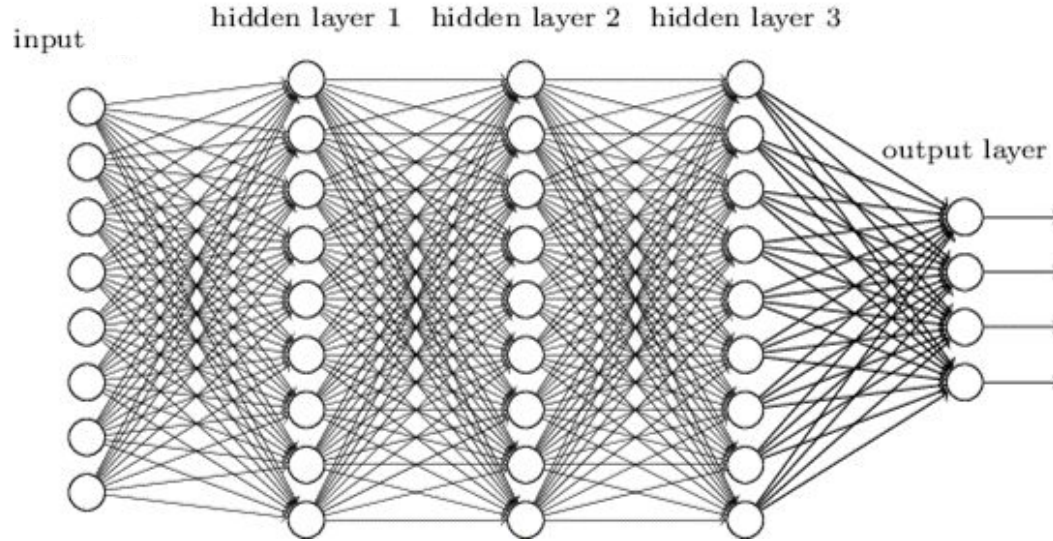


$$\begin{aligned} \mathbf{z}_1 &= \mathbf{W}_1^T \mathbf{x} \\ \mathbf{h}_1 &= \text{sigmoid}(\mathbf{z}_1) \\ \mathbf{z}_2 &= \mathbf{W}_2^T \mathbf{h}_1 \\ \mathbf{h}_2 &= \text{sigmoid}(\mathbf{z}_2) \\ \mathbf{z}_3 &= \mathbf{W}_3^T \mathbf{h}_2 \\ \hat{\mathbf{y}} &= \text{sigmoid}(\mathbf{z}_3) \end{aligned}$$

“Block View” Of MLP



“Deep” Neural Networks (i.e. > 1 hidden layer)



Researchers have successfully used 1000 layers to train an object classifier

Neurons

1-Layer Neural Network

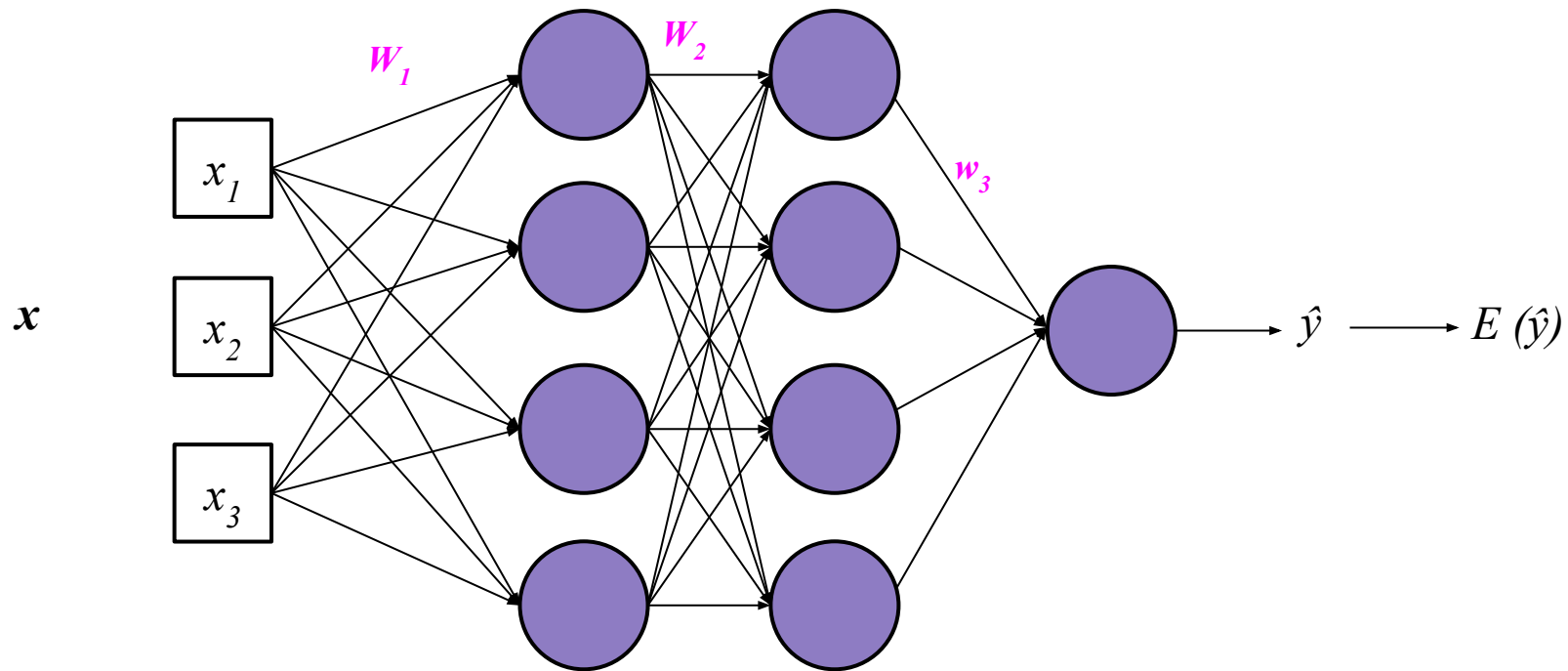
Multi-layer Neural Network

Loss Functions

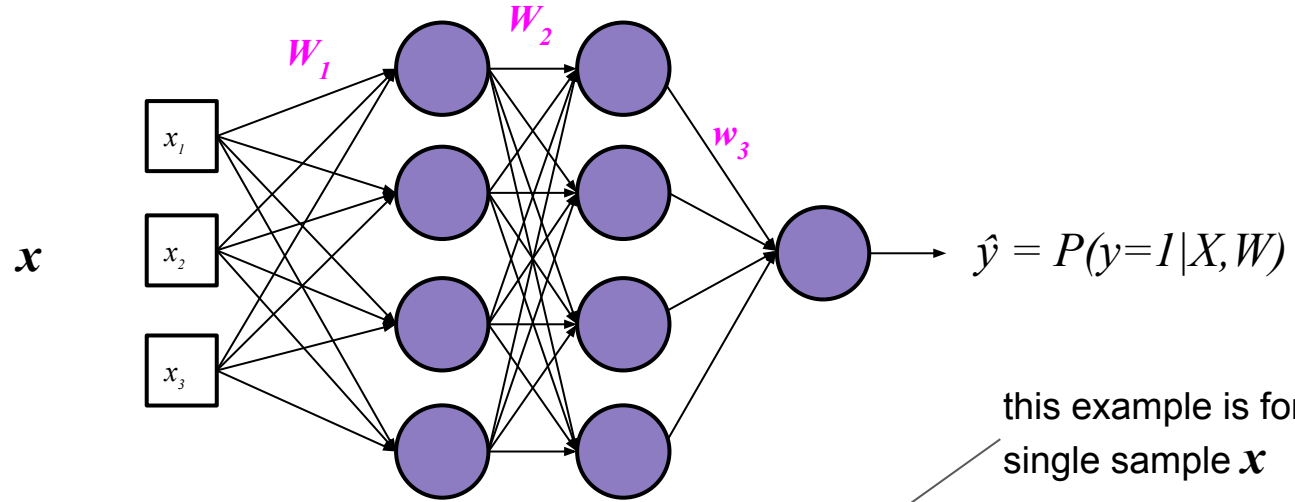
Backpropagation

Nonlinearity Functions

NNs in Practice



Binary Classification Loss

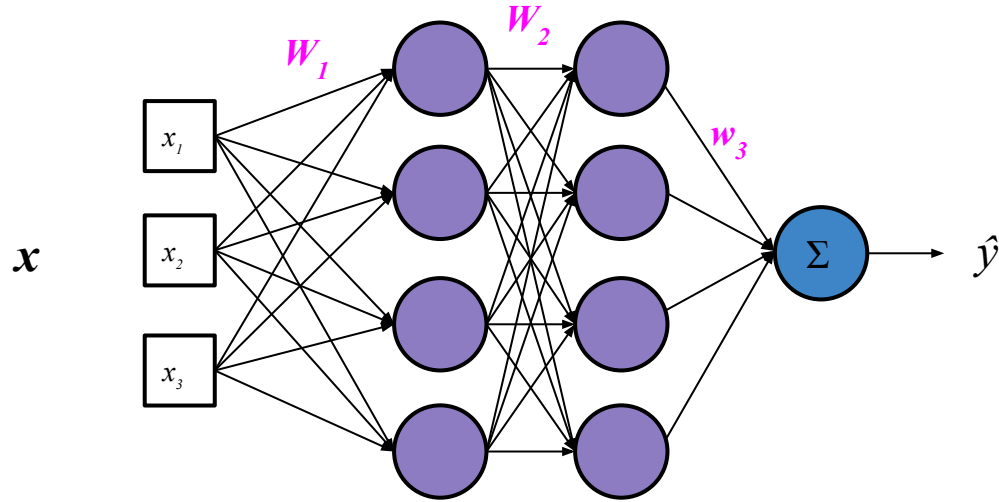


this example is for a
single sample $\mathbf{x}$

$$\begin{aligned} E = \text{loss} &= -\log P(Y = \hat{y} \mid \mathbf{X} = \mathbf{x}) \\ &= -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) \end{aligned}$$

true output

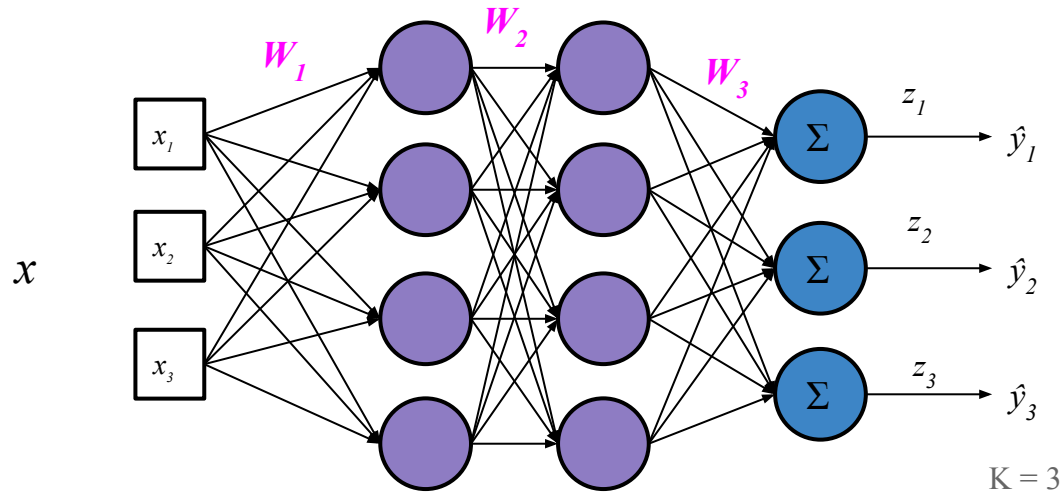
Regression Loss



$$E = \text{loss} = \frac{1}{2} (y - \hat{y})^2$$

true output

Multi-Class Classification Loss



$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = P(\hat{y}_i = I | \mathbf{x})$$

“Softmax” function.

Normalizing function which converts each class output to a probability.

$$E = \text{loss} = - \sum_{j=1 \dots K} y_j \ln \hat{y}_j$$

“0” for all except true class

Neurons

1-Layer Neural Network

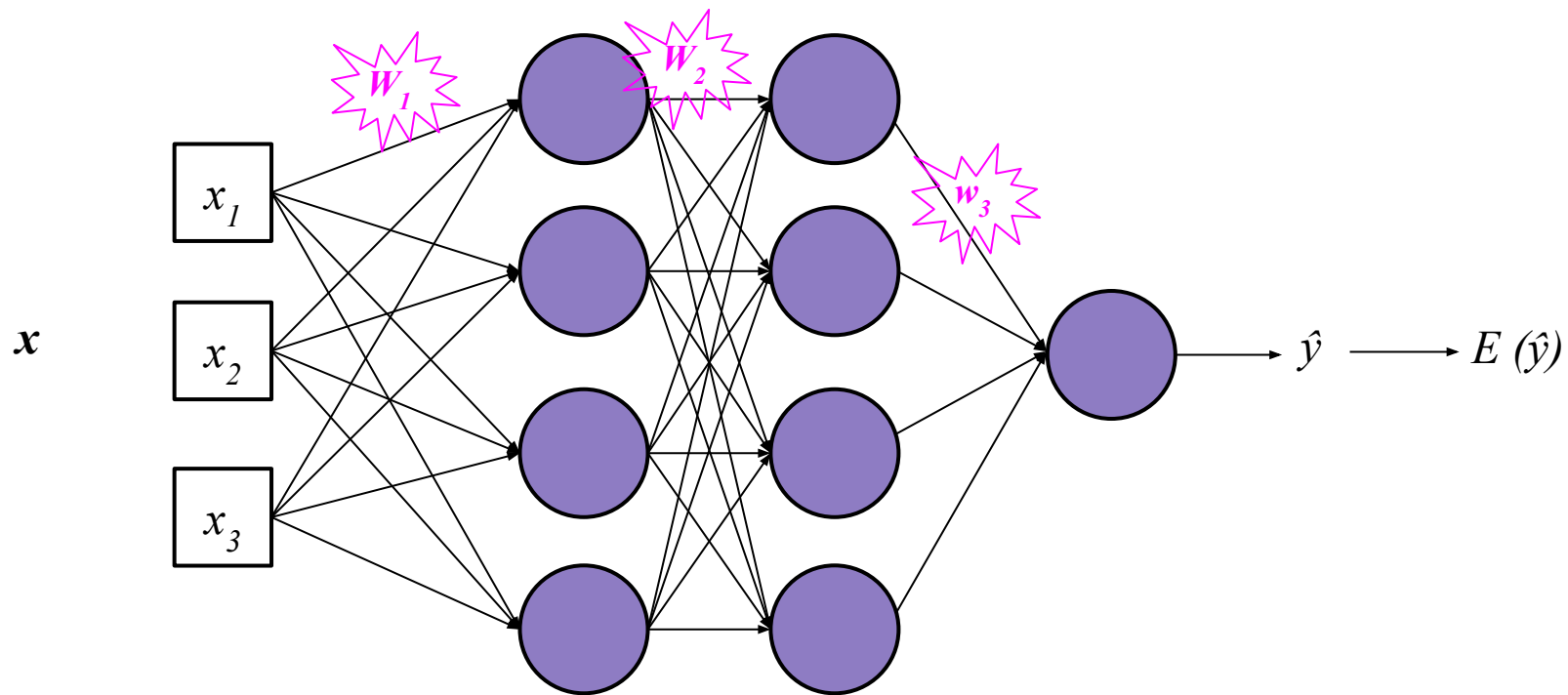
Multi-layer Neural Network

Loss Functions

Backpropagation

Nonlinearity Functions

NNs in Practice

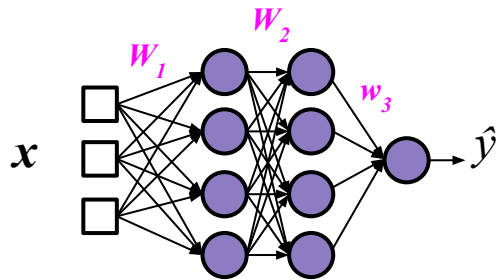


Training Neural Networks

How do we learn the optimal weights W_L for our task??

- **Gradient descent:**

$$W_L(t+1) = W_L(t) - \eta \frac{\partial E}{\partial W_L(t)}$$



But how do we get gradients of lower layers?

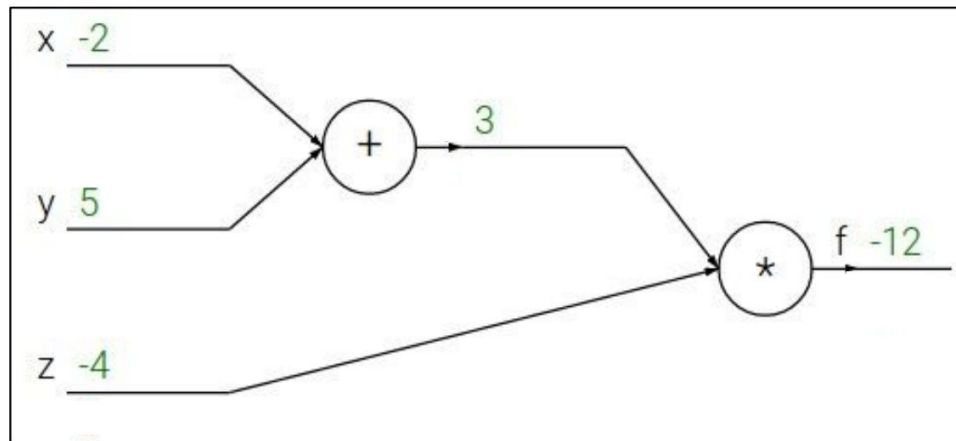
- **Backpropagation!**

- Repeated application of chain rule of calculus
- Locally minimize the objective
- Requires all “blocks” of the network to be differentiable

Backpropagation Intro

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



Backpropagation Intro

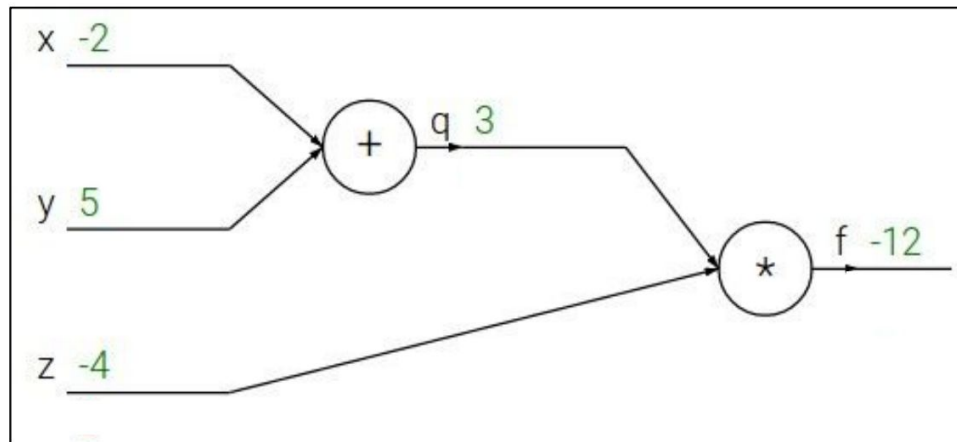
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation Intro

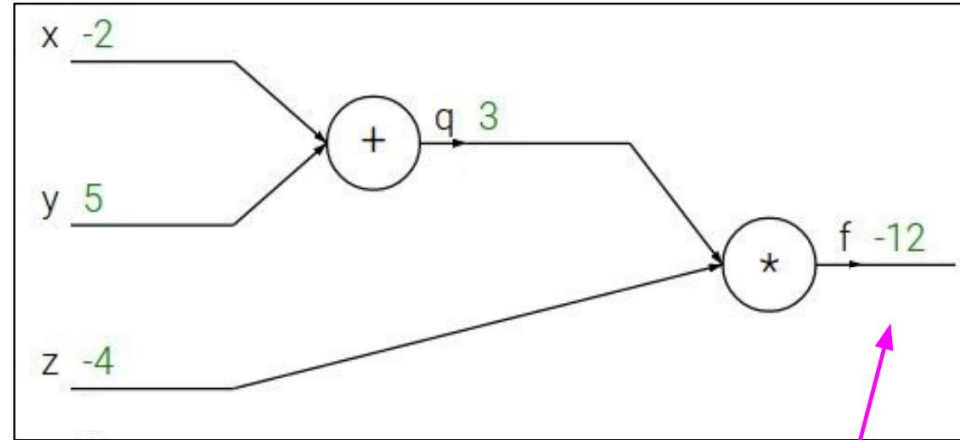
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Backpropagation Intro

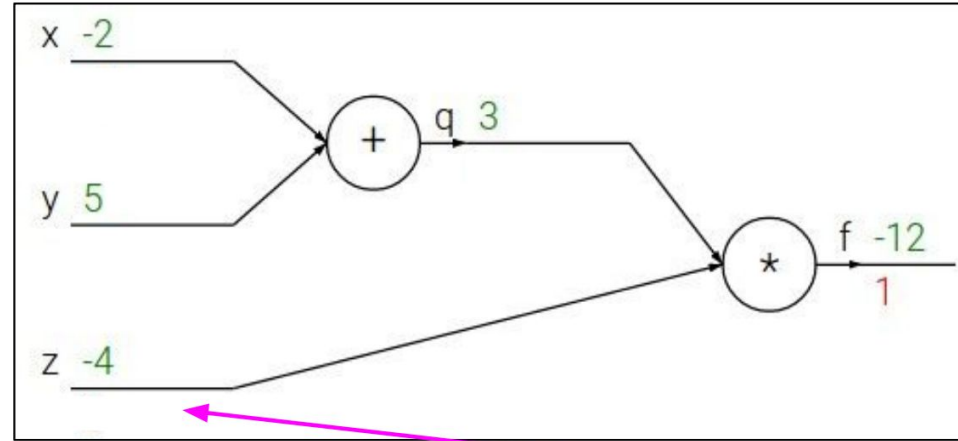
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation Intro

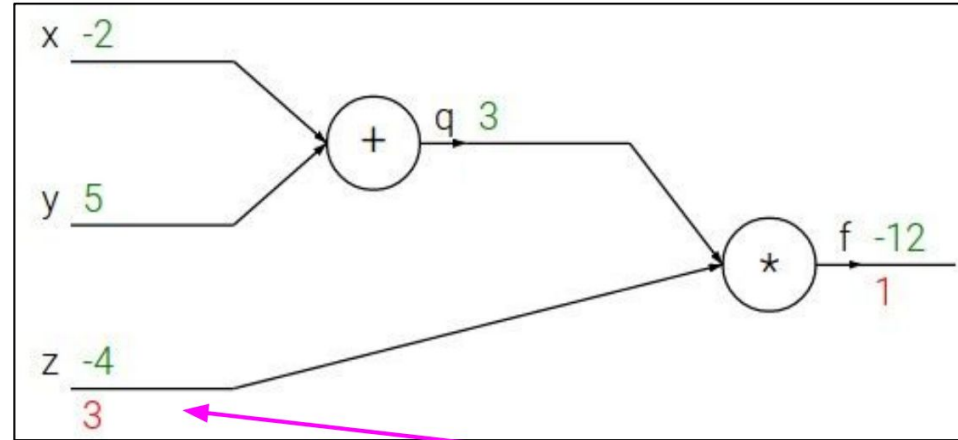
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backpropagation Intro

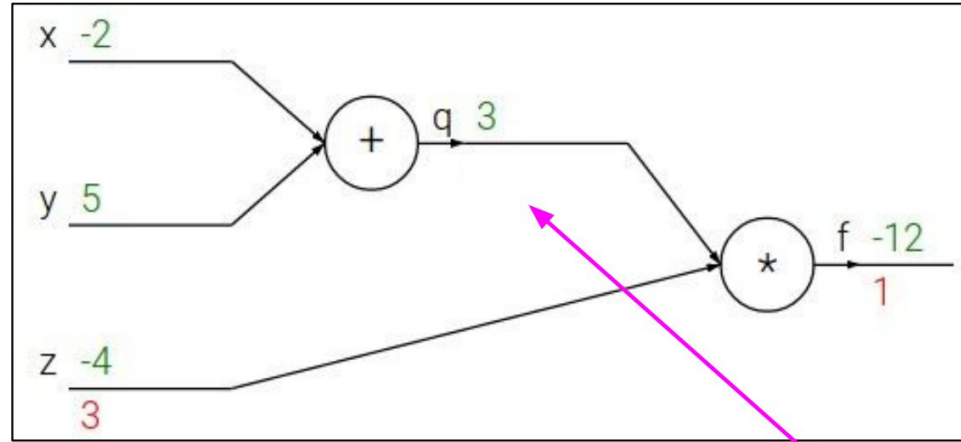
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Backpropagation Intro

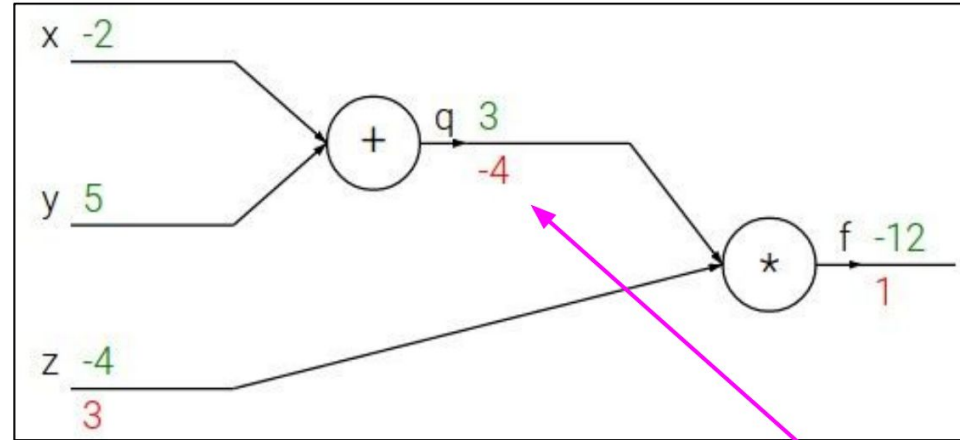
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Backpropagation Intro

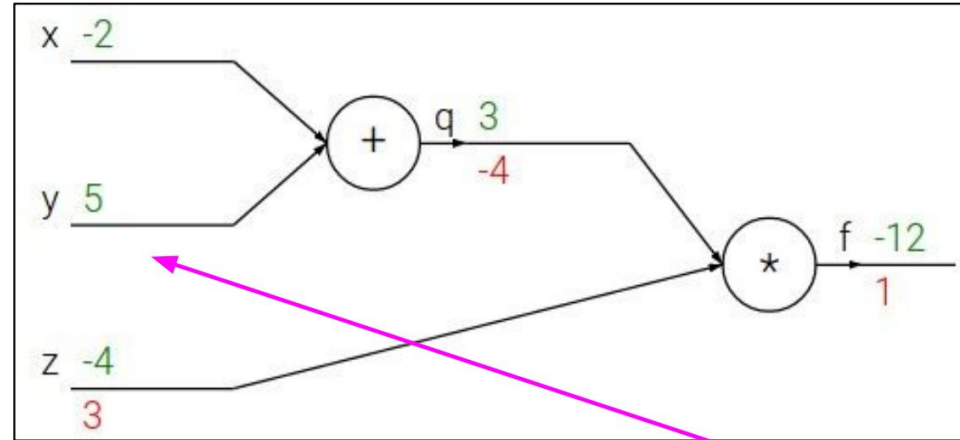
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Backpropagation Intro

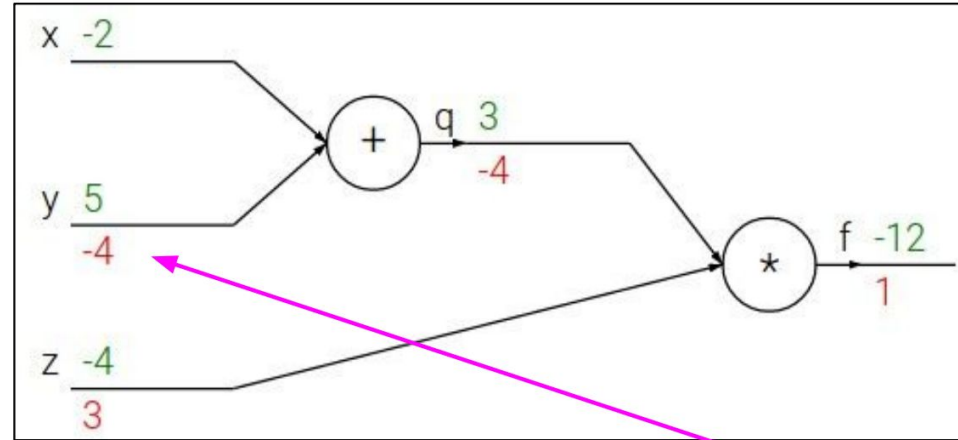
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Backpropagation Intro

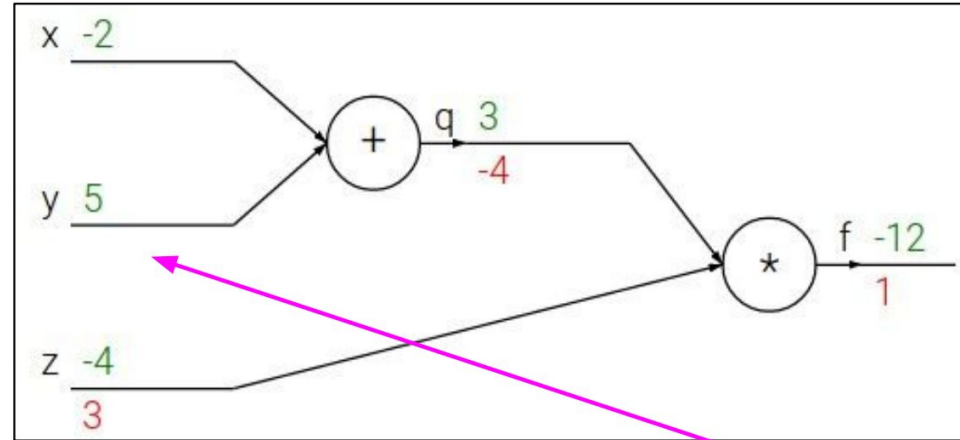
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Backpropagation Intro

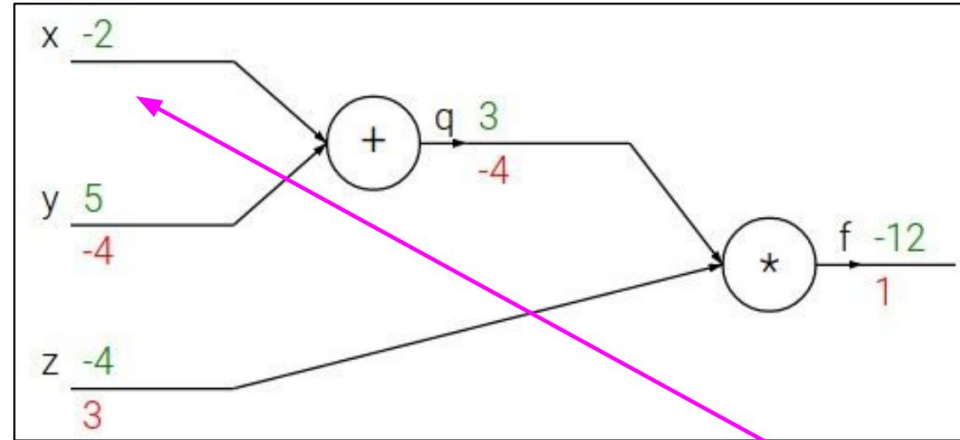
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Backpropagation Intro

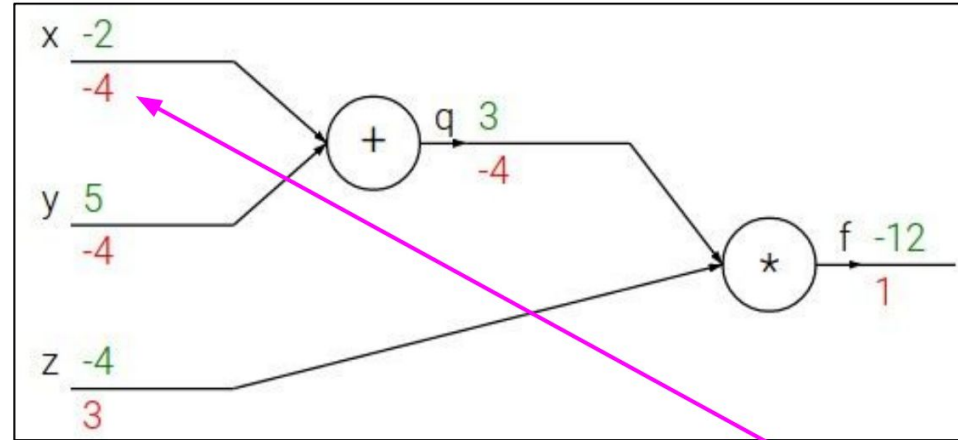
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial x}$$

Backpropagation Intro

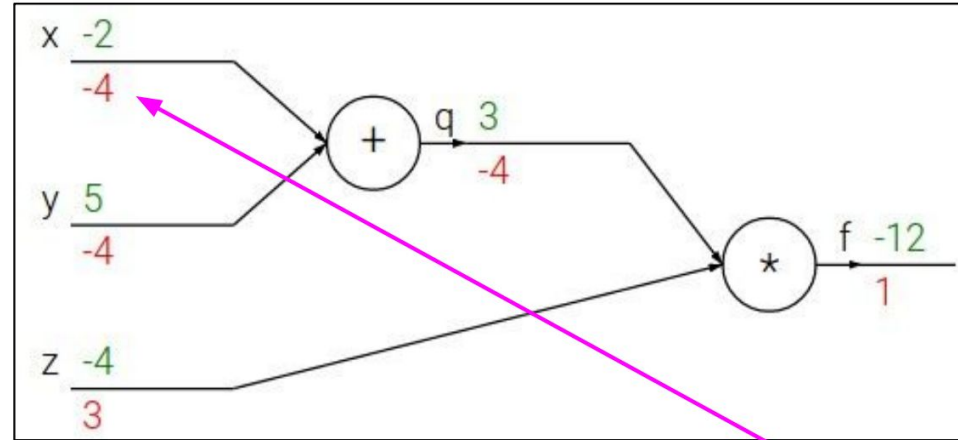
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

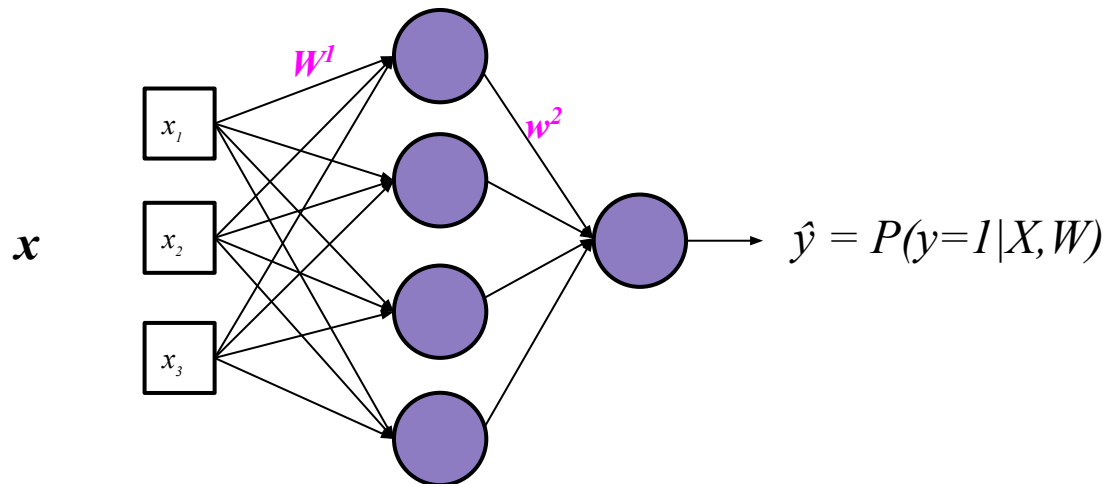


$$\frac{\partial f}{\partial x}$$

Tells us: by increasing x by a scale of 1, we decrease f by a scale of 4

Backpropagation

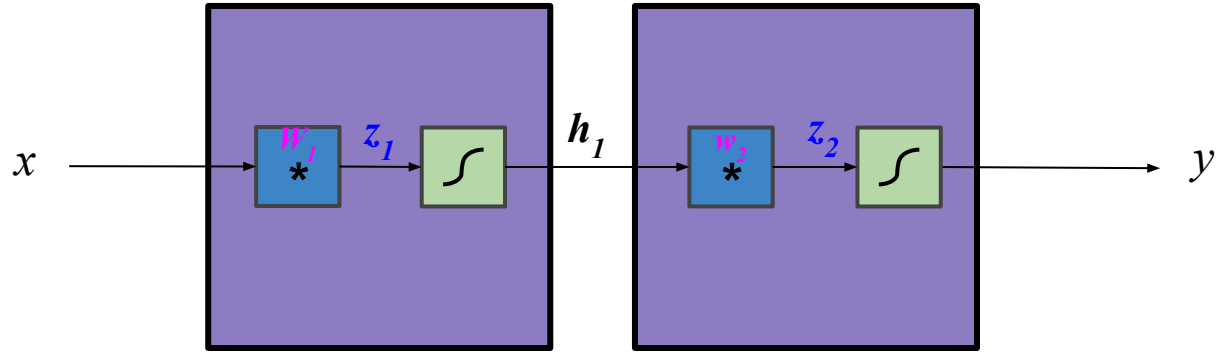
(binary classification example)



Example on 1-hidden layer NN for binary classification

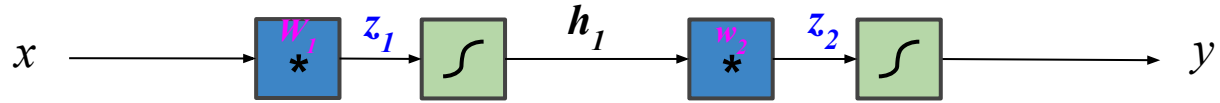
Backpropagation

(binary classification example)



Backpropagation

(binary classification example)



$$E = \text{loss} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

**Gradient Descent
to Minimize loss:**

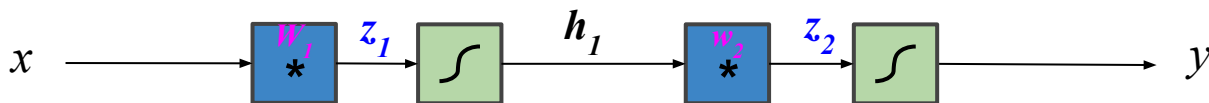
$$\mathbf{w}_2(t + 1) = \mathbf{w}_2(t) - \eta \frac{\partial E}{\partial \mathbf{w}_2(t)}$$

$$W_1(t + 1) = W_1(t) - \eta \frac{\partial E}{\partial W_1(t)}$$

Need to find these!

Backpropagation

(binary classification example)



$$E = f_4 = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = f_3 = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = f_3 = \mathbf{w}_2^T \mathbf{h}_1$$

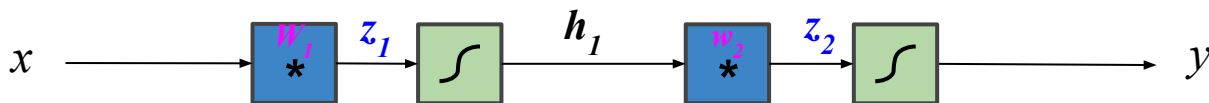
$$\mathbf{h}_1 = f_2 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$z_1 = f_1 = \mathbf{W}_1^T \mathbf{x}$$

$$E = f_4(f_3(f_2(f_1(x))))$$

Backpropagation

(binary classification example)



$$E = f_4 = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = f_3 = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = f_3 = \mathbf{w}_2^T \mathbf{h}_1$$

$$\mathbf{h}_1 = f_2 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$z_1 = f_1 = \mathbf{W}_1^T \mathbf{x}$$

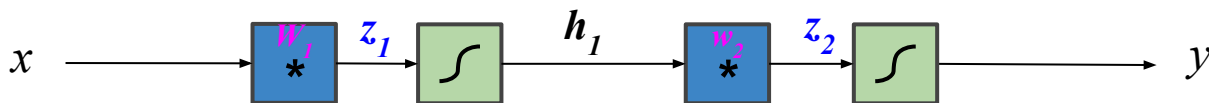
$$\frac{\partial E}{\partial \mathbf{w}_2} = ??$$

$$\frac{\partial E}{\partial \mathbf{W}_1} = ??$$

$$E = f_4(f_3(f_2(f_1(x))))$$

Backpropagation

(binary classification example)



$$E = f_4 = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = f_3 = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = f_3 = \mathbf{w}_2^T \mathbf{h}_1$$

$$\mathbf{h}_1 = f_2 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$z_1 = f_1 = \mathbf{W}_1^T \mathbf{x}$$

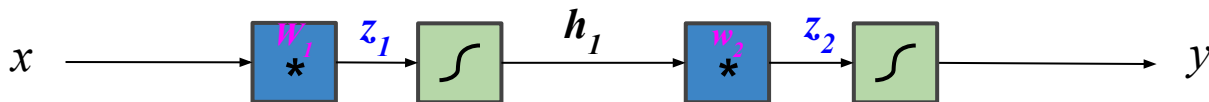
$$\frac{\partial E}{\partial \mathbf{w}_2} = ??$$

$$\frac{\partial E}{\partial \mathbf{W}_1} = ??$$

$E = f_4(f_3(f_2(f_1(x))))$  Exploit the chain rule!

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

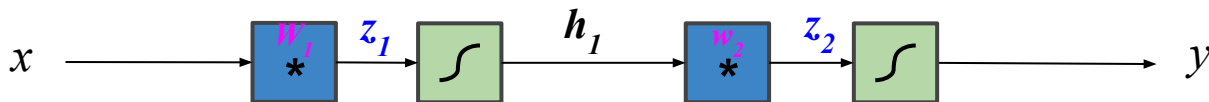
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\frac{\partial E}{\partial w_2} =$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

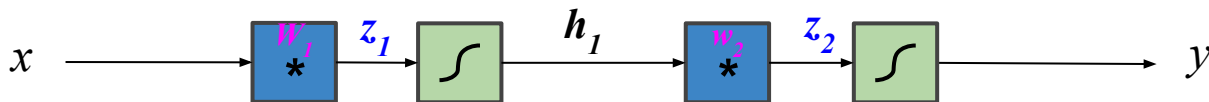
$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

chain rule

$$\frac{\partial E}{\partial \mathbf{w}_2} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{w}_2}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

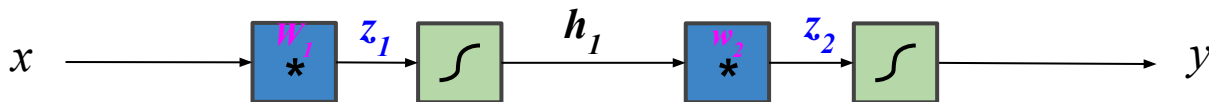
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{w}_2} &= \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{w}_2} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \end{aligned}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

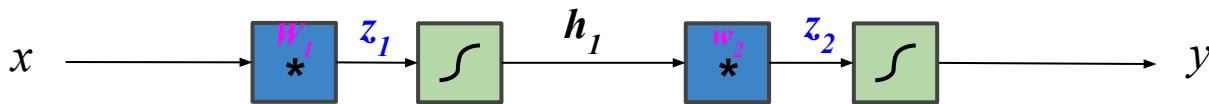
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{w}_2} &= \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{w}_2} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \left(\frac{e^{z_2}}{1 + e^{z_2}} \left(1 - \frac{e^{z_2}}{1 + e^{z_2}} \right) \right) \end{aligned}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

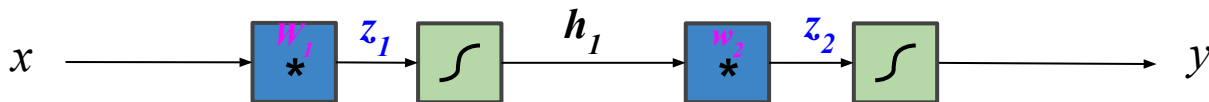
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{w}_2} &= \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{w}_2} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \left(\frac{e^{z_2}}{1 + e^{z_2}} \left(1 - \frac{e^{z_2}}{1 + e^{z_2}} \right) \right) \cdot (\mathbf{h}_1) \end{aligned}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \cdot \mathbf{h}_1$$

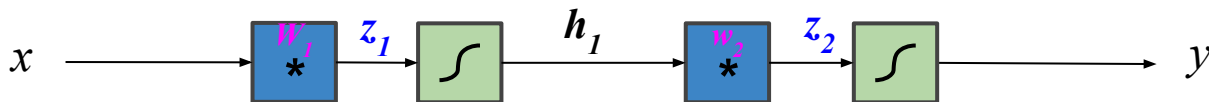
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$z_1 = \mathbf{W}_1^T \cdot \mathbf{x}$$

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{w}_2} &= \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{w}_2} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \left(\frac{e^{z_2}}{1 + e^{z_2}} \left(1 - \frac{e^{z_2}}{1 + e^{z_2}} \right) \right) \cdot (\mathbf{h}_1) \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot (\hat{y}(1 - \hat{y})) \cdot (\mathbf{h}_1) \end{aligned}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y})$$

$$- (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

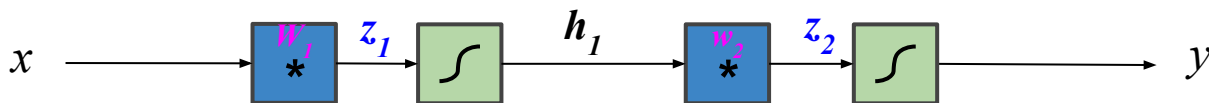
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\frac{\partial E}{\partial \mathbf{W}_1} =$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y})$$

$$- (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

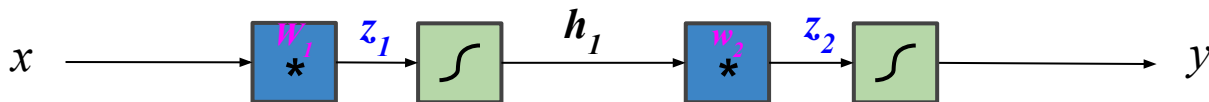
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\frac{\partial E}{\partial \mathbf{W}_1} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial \mathbf{W}_1}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

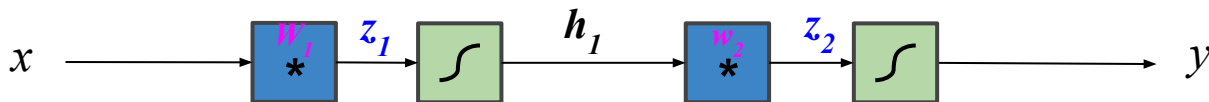
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{W}_1} &= \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial \mathbf{W}_1} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot (\hat{y}(1 - \hat{y})) \cdot (\mathbf{w}) \cdot (\mathbf{h}_1(1 - \mathbf{h}_1)) \cdot (\mathbf{x}) \end{aligned}$$

Backpropagation

(binary classification example)



$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

$$\hat{y} = \frac{e^{z_2}}{1 + e^{z_2}}$$

$$z_2 = \mathbf{w}_2^T \mathbf{h}_1$$

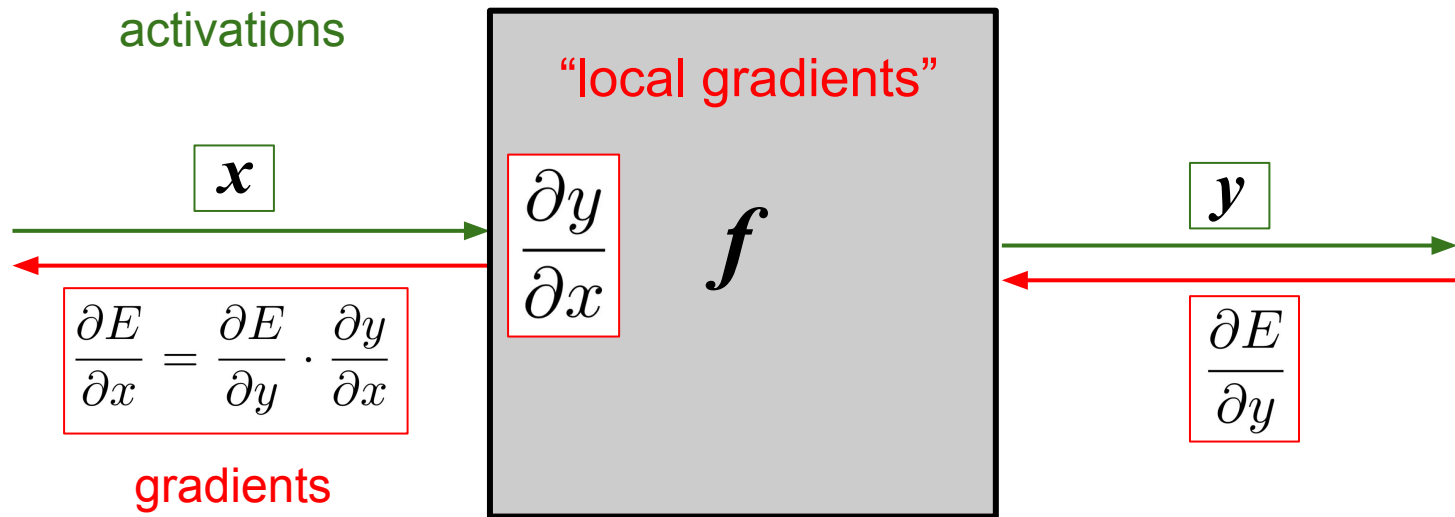
$$\mathbf{h}_1 = \frac{e^{z_1}}{1 + e^{z_1}}$$

$$\mathbf{z}_1 = \mathbf{W}_1^T \mathbf{x}$$

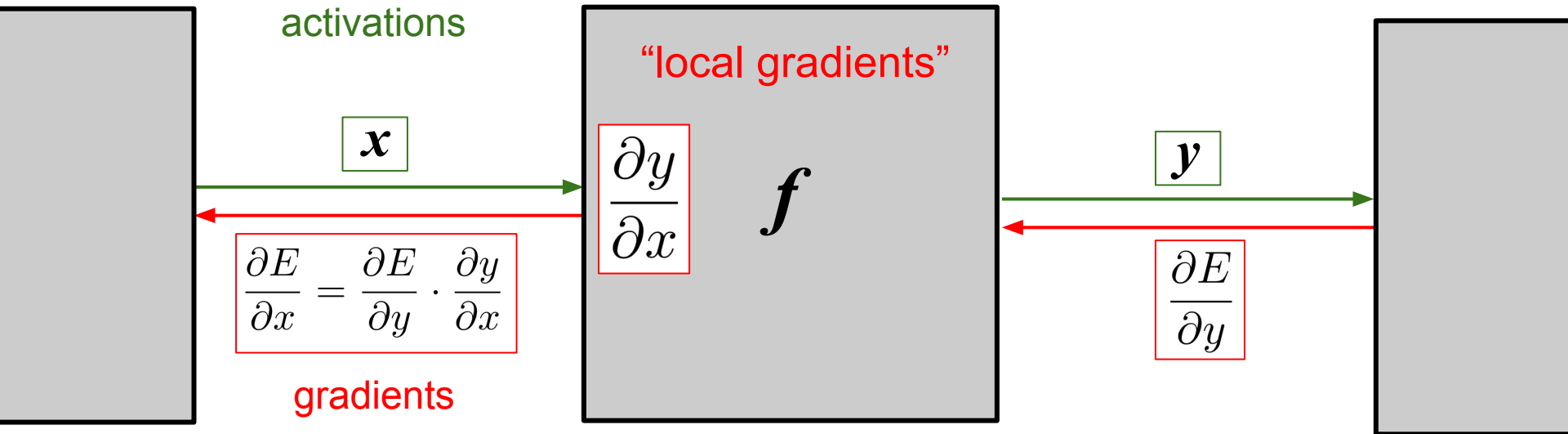
already computed

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{W}_1} &= \boxed{\frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2}} \cdot \frac{\partial z_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial \mathbf{W}_1} \\ &= \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot (\hat{y}(1 - \hat{y})) \cdot (\mathbf{w}) \cdot (\mathbf{h}_1(1 - \mathbf{h}_1)) \cdot (\mathbf{x}) \end{aligned}$$

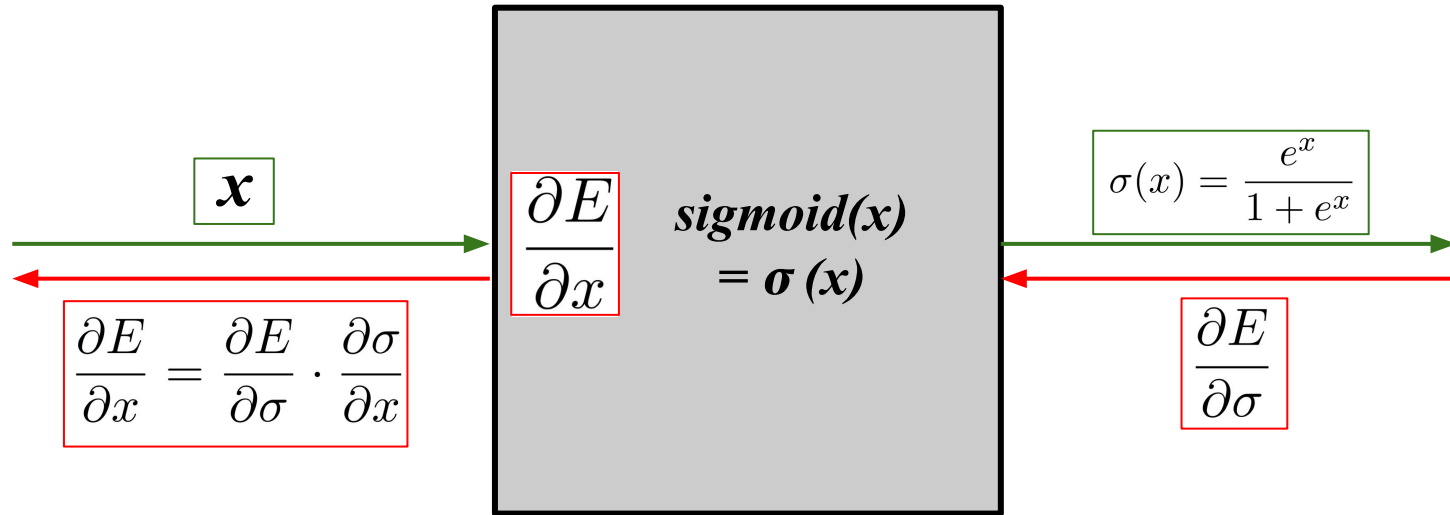
“Local-ness” of Backpropagation



“Local-ness” of Backpropagation

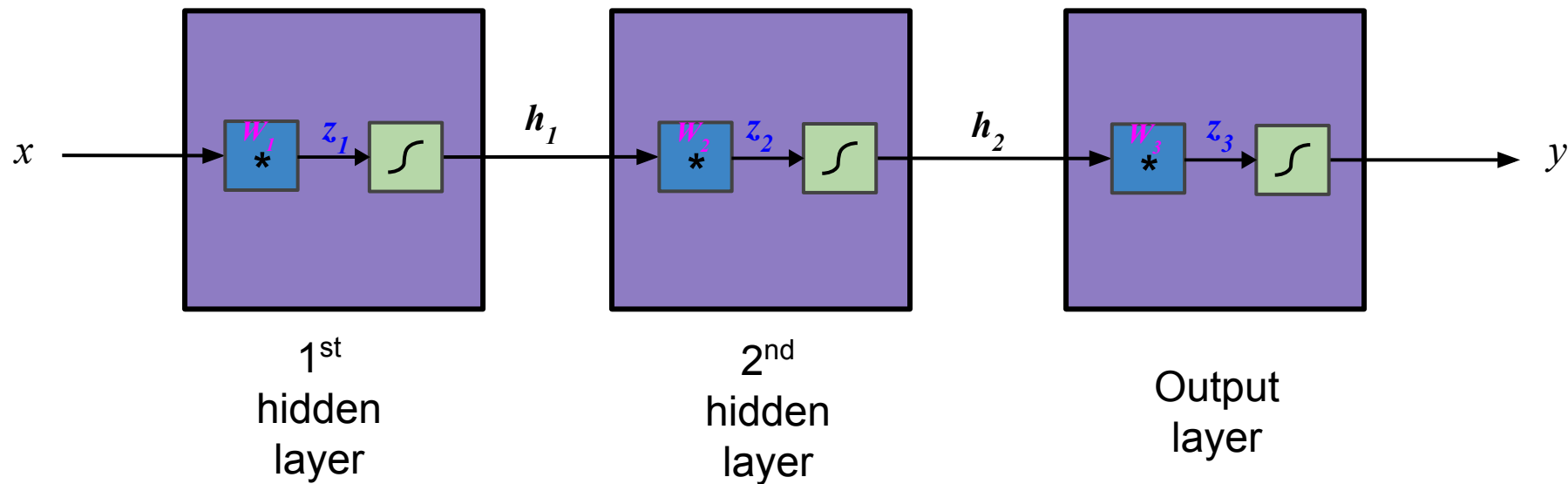


Example: Sigmoid Block



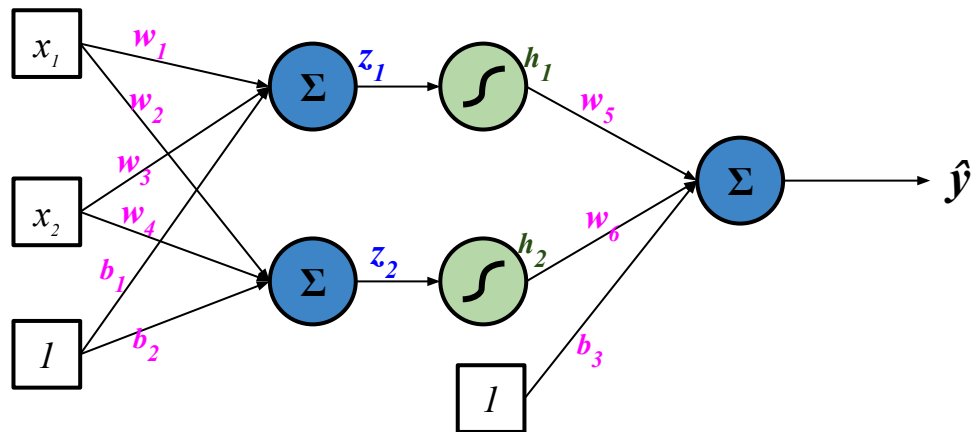
Deep Learning =

Concatenation of Differentiable Parameterized Layers (linear & nonlinearity functions)



Want to find optimal weights w to minimize some loss function E !

Backprop Whiteboard Demo



f_1

$$z_1 = x_1 w_1 + x_2 w_3 + b_1$$

$$z_2 = x_1 w_2 + x_2 w_4 + b_2$$

f_2

$$h_1 = \frac{\exp(z_1)}{1 + \exp(z_1)}$$

$$h_2 = \frac{\exp(z_2)}{1 + \exp(z_2)}$$

f_3

$$\hat{y} = h_1 w_5 + h_2 w_6 + b_3$$

f_4

$$E = (y - \hat{y})^2$$

$$w(t+1) = w(t) - \eta \frac{\partial E}{\partial w(t)}$$

$$\frac{\partial E}{\partial w} = ??$$

Neurons

1-Layer Neural Network

Multi-layer Neural Network

Loss Functions

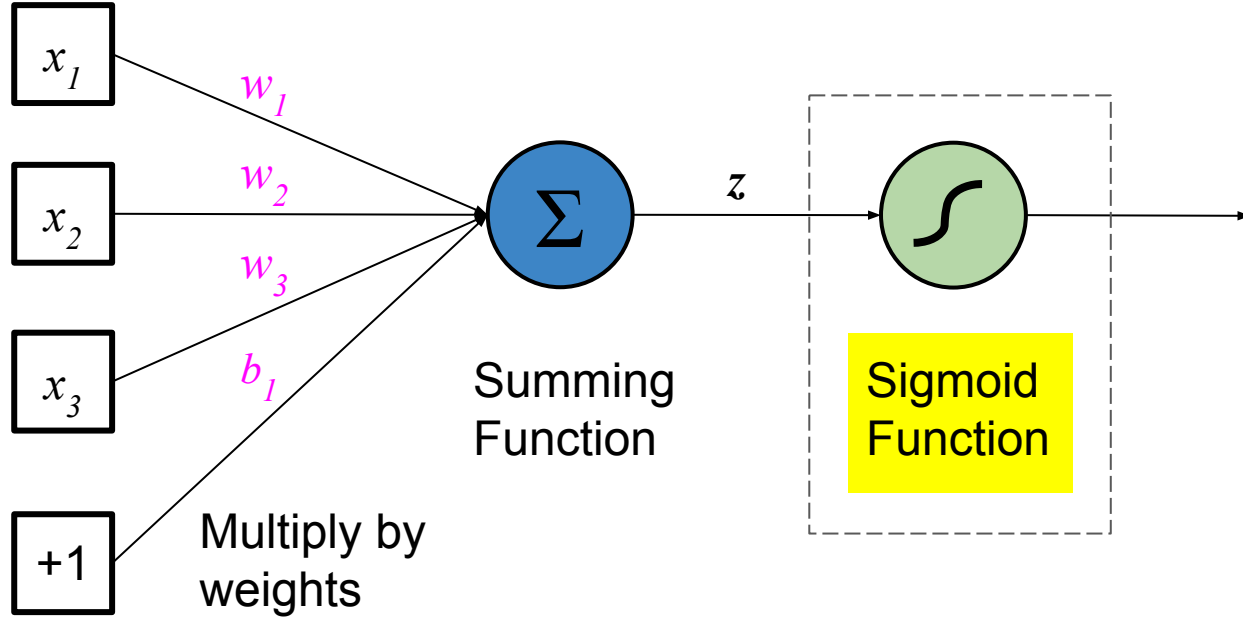
Backpropagation

Nonlinearity Functions

NNs in Practice

Nonlinearity Functions

(i.e. transfer or activation functions)



Nonlinearity Functions

(i.e. transfer or activation functions)

Name	Plot	Equation	Derivative (w.r.t x)
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
Rectifier (ReLU) ^[9]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Nonlinearity Functions

(i.e. transfer or activation functions)

Name	Plot	Equation	Derivative (w.r.t x)
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
Rectifier (ReLU) ^[9]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Nonlinearity Functions

(aka transfer or activation functions)

Name	Plot	Equation	Derivative (w.r.t x)
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
Rectifier (ReLU) ^[9]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

usually works best in practice

Neurons

1-Layer Neural Network

Multi-layer Neural Network

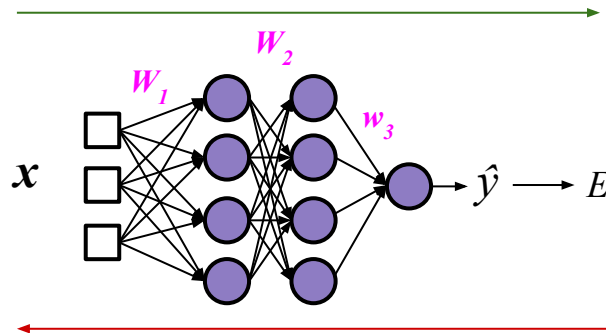
Loss Functions

Backpropagation

Nonlinearity Functions

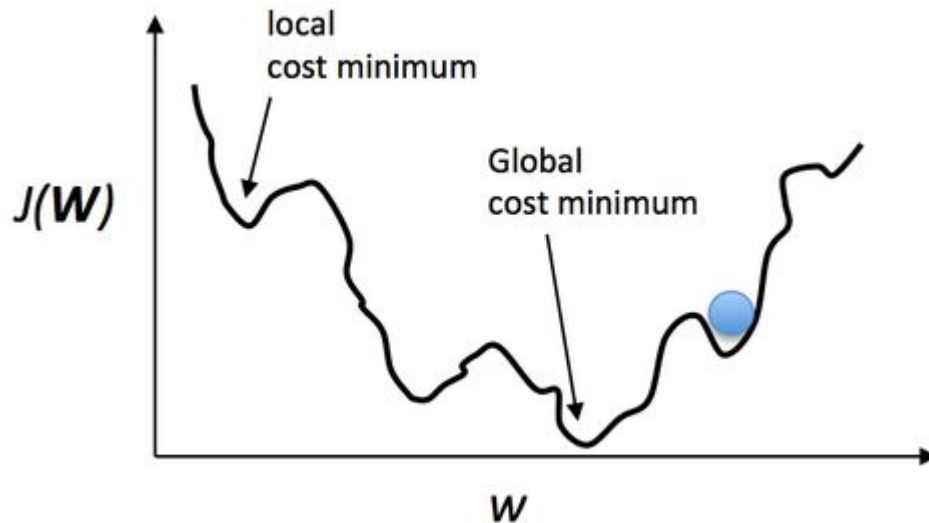
NNs in Practice

Neural Net Pipeline



1. Initialize weights
2. For each batch of input x samples S :
 - a. Run the network “**Forward**” on S to compute outputs and loss
 - b. Run the network “**Backward**” using outputs and loss to compute gradients
 - c. Update weights using SGD (or a similar method)
3. Repeat step 2 until loss convergence

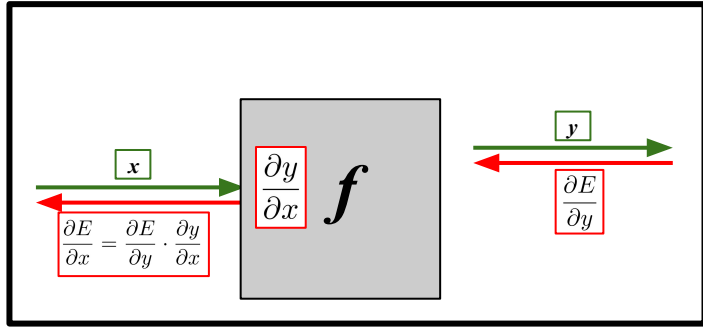
Non-Convexity of Neural Nets



In very high dimensions, there exists many local minimum which are about the same.

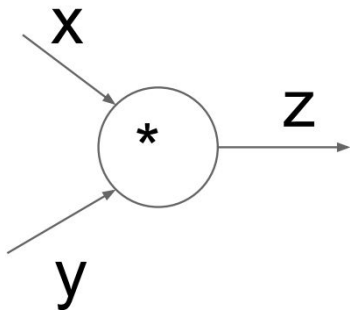
Pascanu, et. al. *On the saddle point problem for non-convex optimization* 2014

Building Deep Neural Nets



77

Block Example Implementation

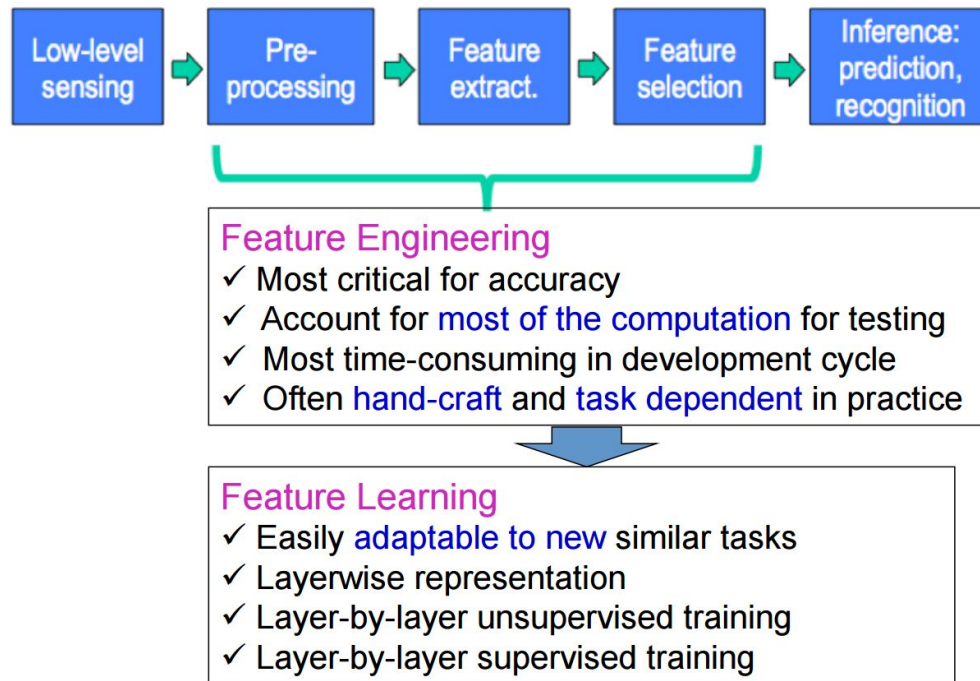


(x,y,z are scalars)

```
class MultiplyGate(object):  
    def forward(x,y):  
        z = x*y  
        self.x = x # must keep these around!  
        self.y = y  
        return z  
    def backward(dz):  
        dx = self.y * dz # [dz/dx * dL/dz]  
        dy = self.x * dz # [dz/dy * dL/dz]  
        return [dx, dy]
```



Advantage of Neural Nets



As long as it's fully differentiable, we can train the model to automatically learn features for us.